

Data Communications and Networking

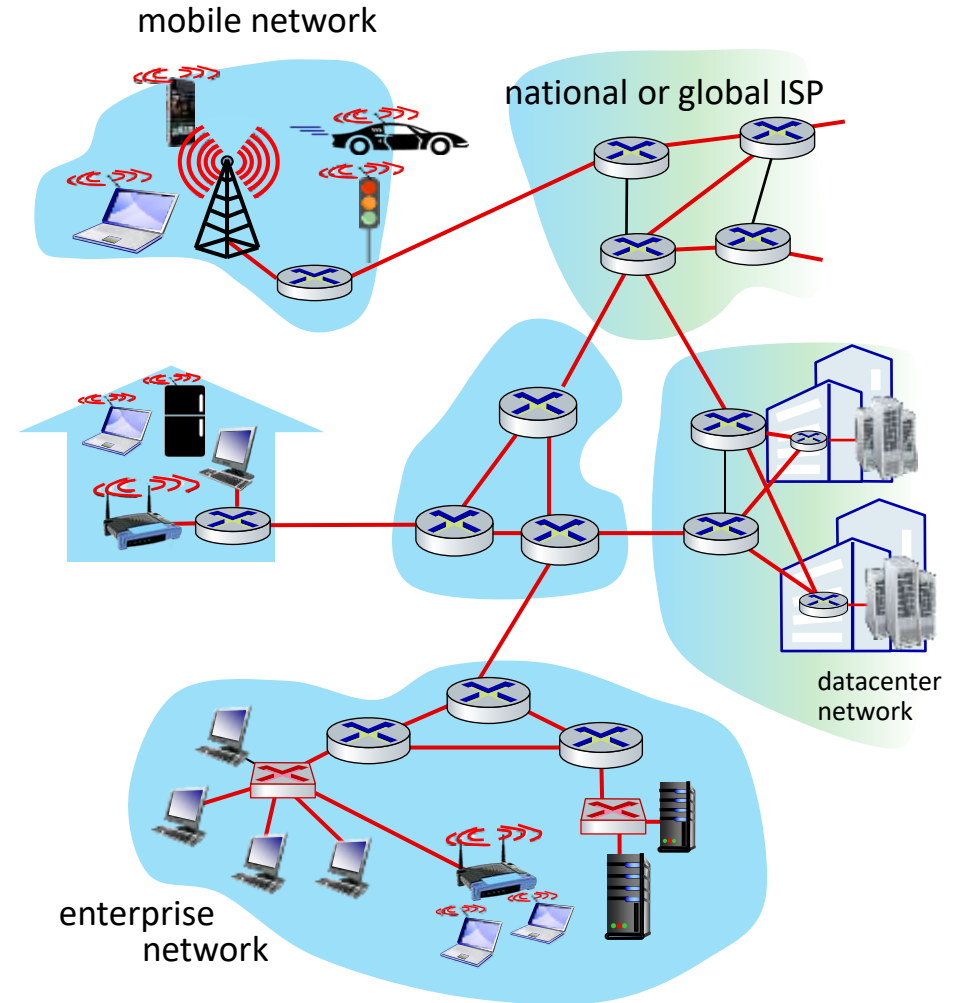
Chapter 4: Data Link Layer

Data Link Layer: Introduction

Terminology:

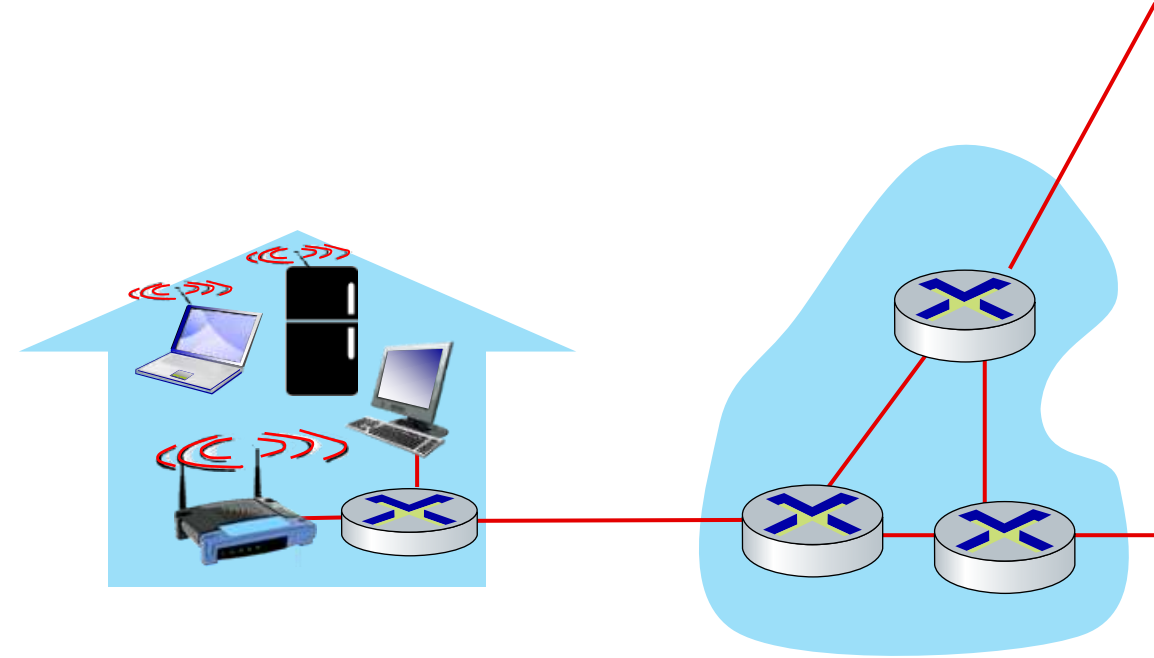
- hosts, routers: **nodes**
- communication channels that connect **adjacent** nodes along communication path: **links**
 - wired , wireless
 - LANs
- layer-2 packet: **frame**, encapsulates datagram

*link layer has responsibility of transferring datagram from one node to **physically adjacent** node over a link*



Data Link Layer: Context

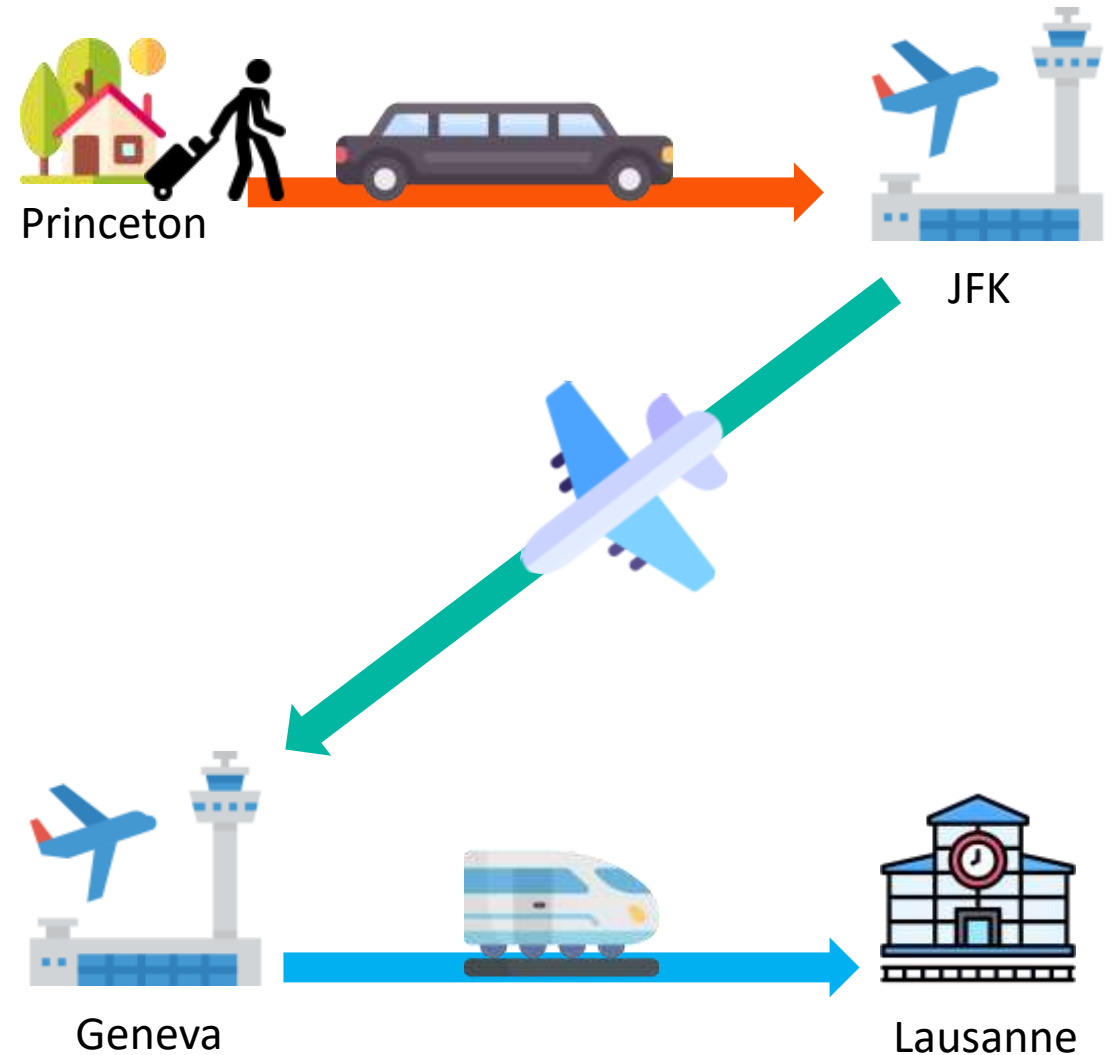
- ❑ datagram transferred by **different link protocols** over different links:
 - e.g., WiFi on first link, Ethernet on next link
- ❑ each link protocol provides different services
 - e.g., **may or may not** provide reliable data transfer over link



Transportation Analogy

□ Transportation analogy:

- trip from Princeton to Lausanne
 - limo: Princeton to JFK
 - plane: JFK to Geneva
 - train: Geneva to Lausanne
- tourist = **datagram**
- transport segment = **communication link**
- transportation mode = **link-layer protocol**
- travel agent = **routing algorithm**



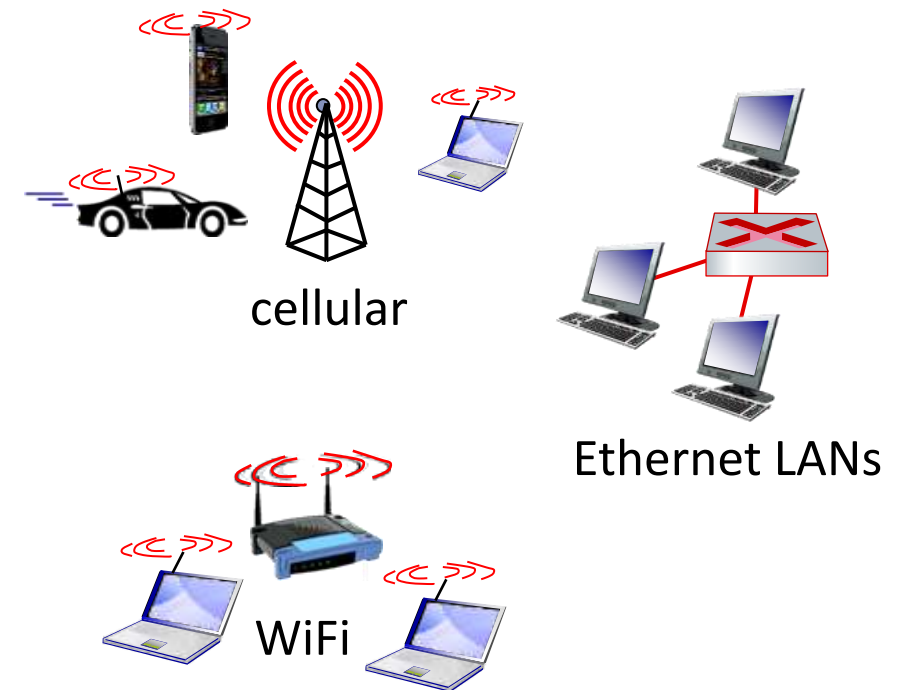
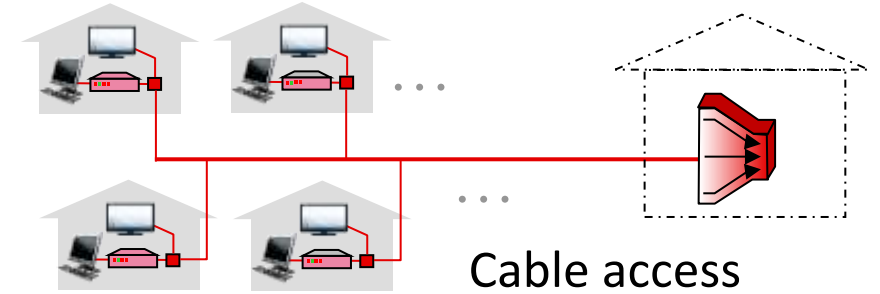
Link Layer Services

□ Framing, link access:

- encapsulate datagram into frame, adding header, trailer
- channel access if shared medium
- “MAC” addresses in frame headers identify source, destination (different from IP address!)

□ Reliable delivery between adjacent nodes

- we already know how to do this!
- seldom used on low bit-error links
- wireless links: high error rates
 - Q: why both link-level and end-end reliability?



Link Layer Services (more)

❑ Flow Control

- pacing between adjacent sending and receiving nodes

❑ Error Detection

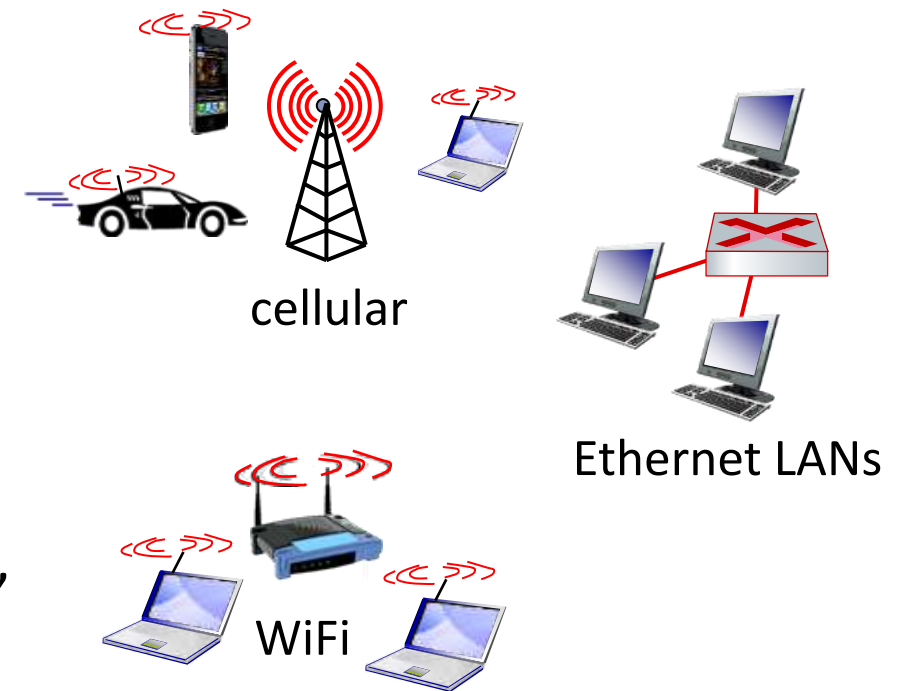
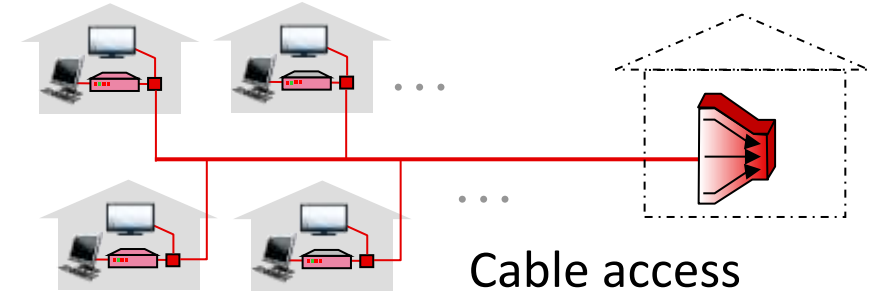
- errors caused by signal attenuation, noise.
- receiver detects errors, signals retransmission, or drops frame

❑ Error Correction

- receiver identifies *and corrects* bit error(s) without retransmission

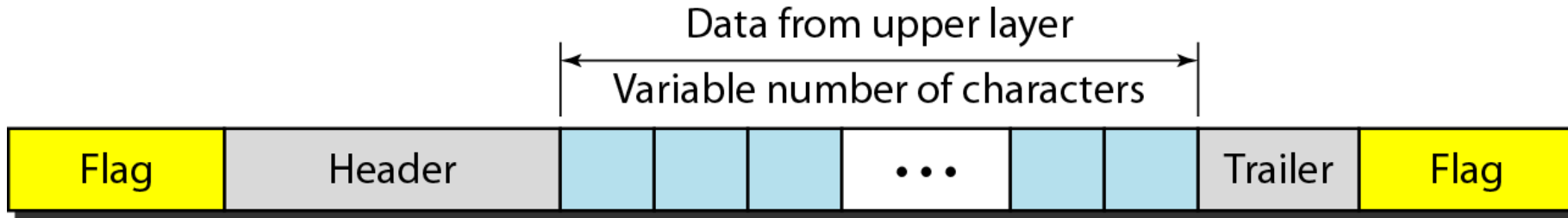
❑ Half-duplex and Full-duplex

- with half duplex, nodes at both ends of link can transmit, but not at same time



Framing

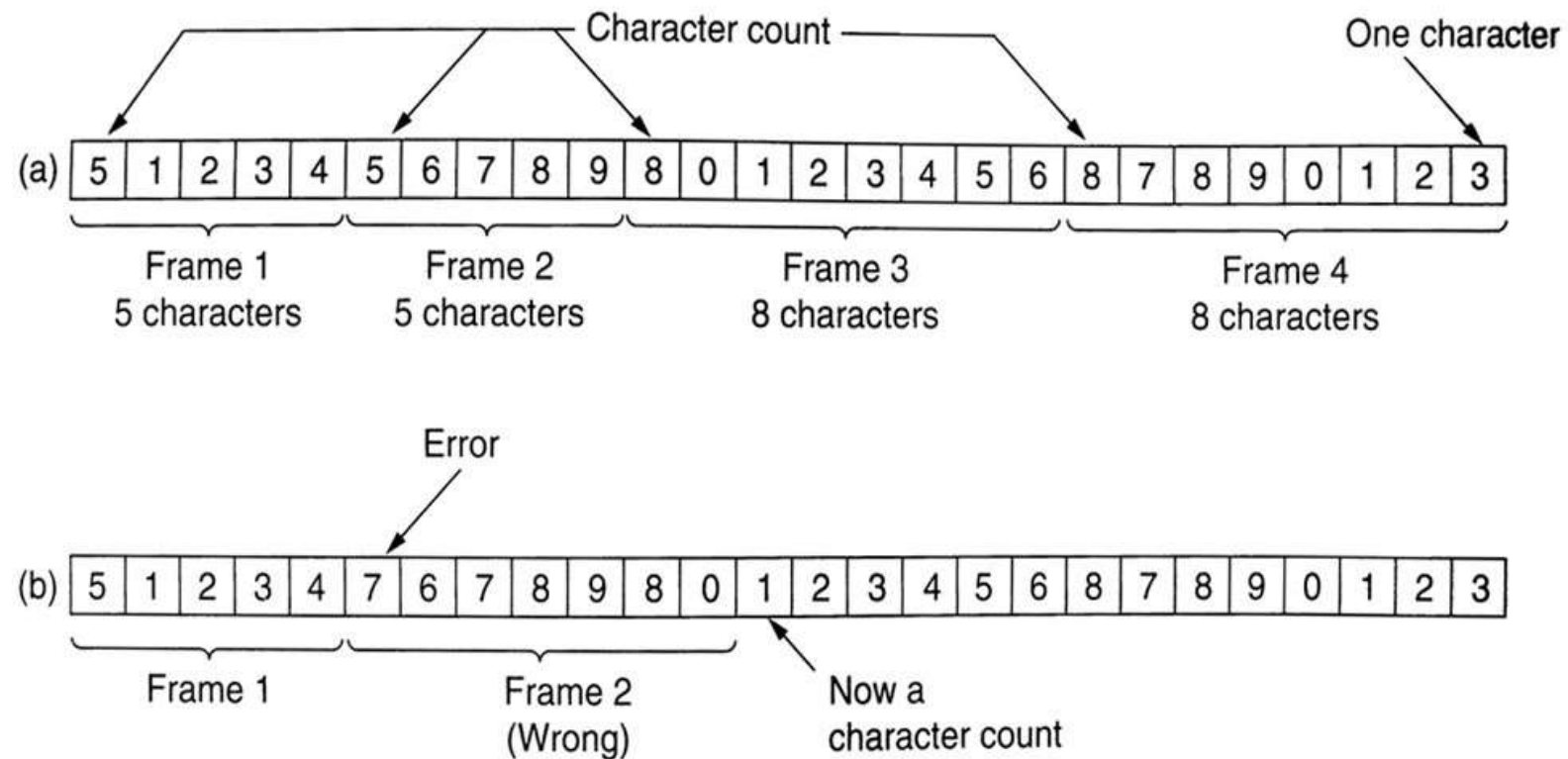
- ❑ The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another.
- ❑ Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.



A frame in a character-oriented protocol

Framing: Character count

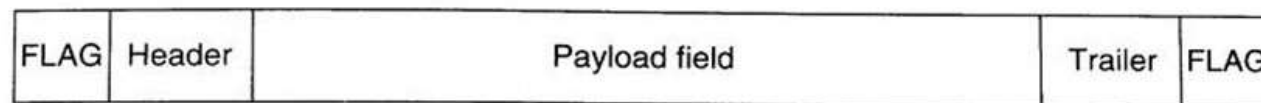
- ❑ Use a field in the header to specify the number of the characters in the frame.



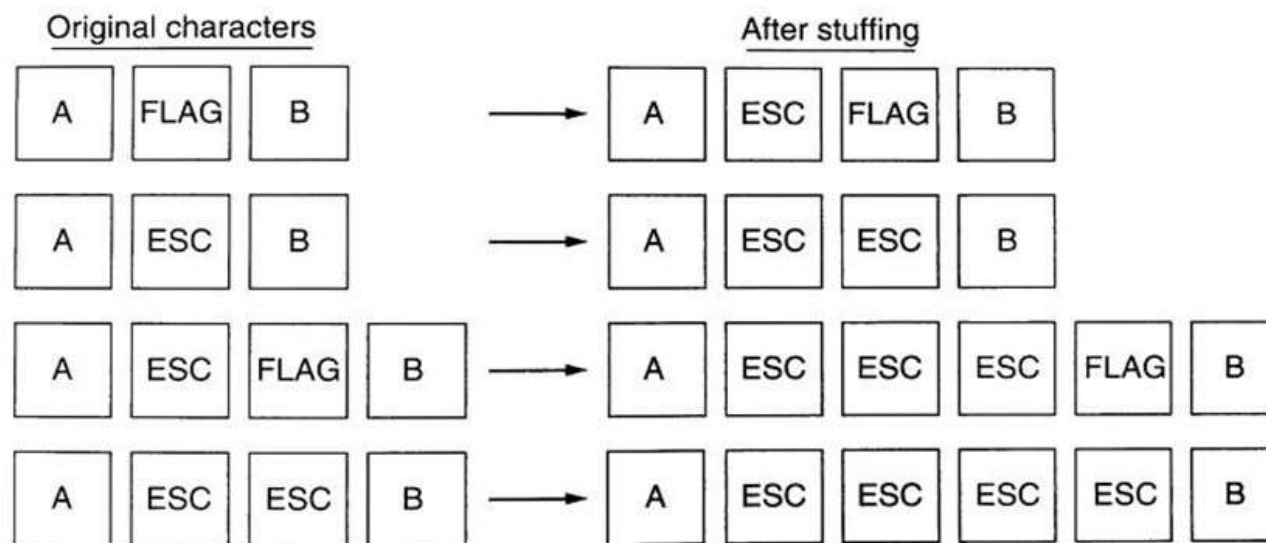
Flag Bytes with Byte Stuffing



- Each frame starts and ends with special bytes. Byte stuffing is the process of adding **1 extra byte whenever** there is a **flag or escape** character in the text.



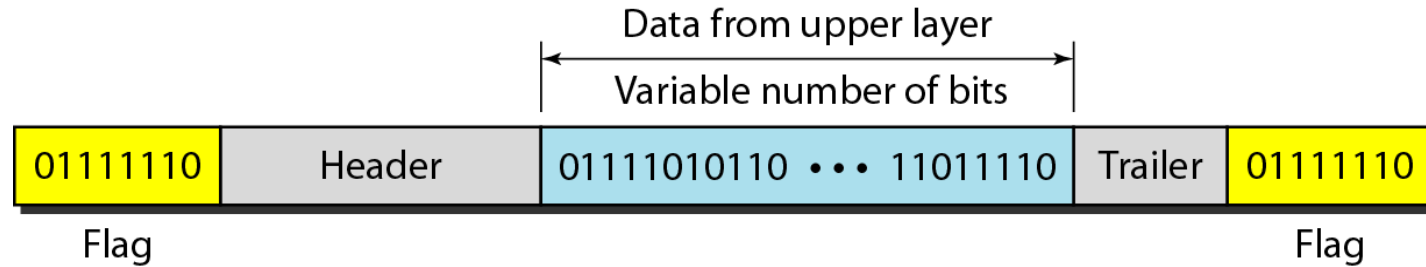
(a)



(b)

Bit Stuffing ★

- Each frame begins and ends with a special bit pattern, 01111110

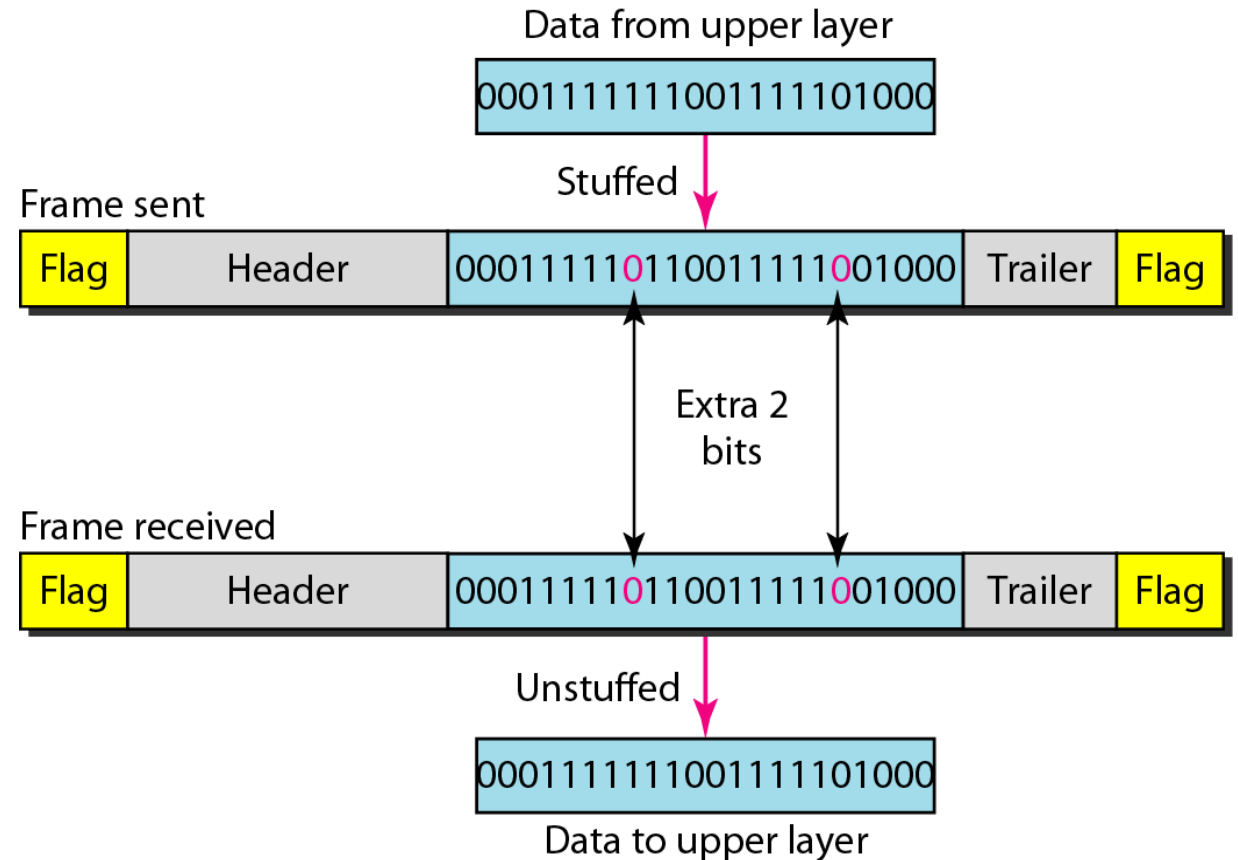


A frame in a bit-oriented protocol

Q: What if the data contains “01111110”?

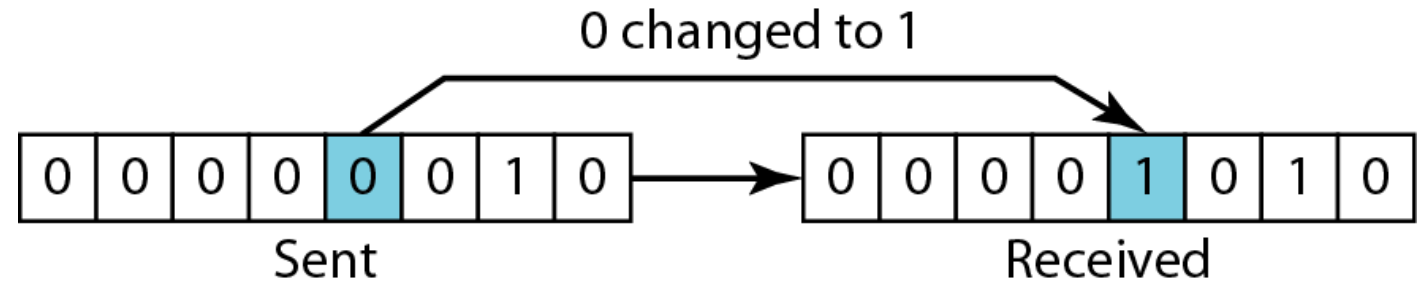
Bit Stuffing ★

□ **Bit stuffing** is the process of adding **one extra 0** whenever **five consecutive 1s** in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

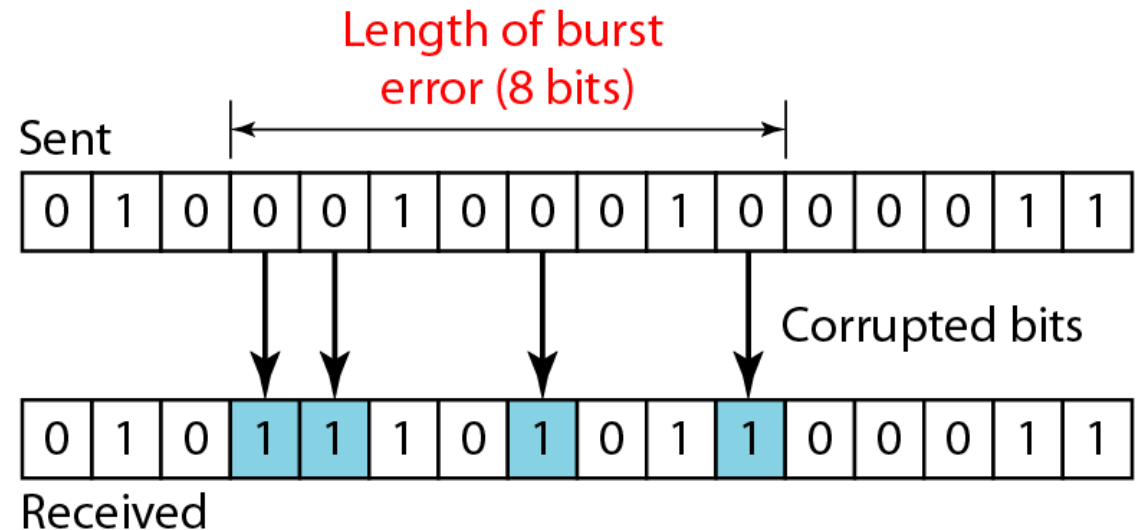


Type of Error

❑ In a **single-bit error**, only 1 bit in the data unit has changed



❑ A **burst error** means that 2 or more bits in the data unit have changed



Detection versus Correction

❑ **The correction of errors is more difficult than the detection**

❑ **Error detection**

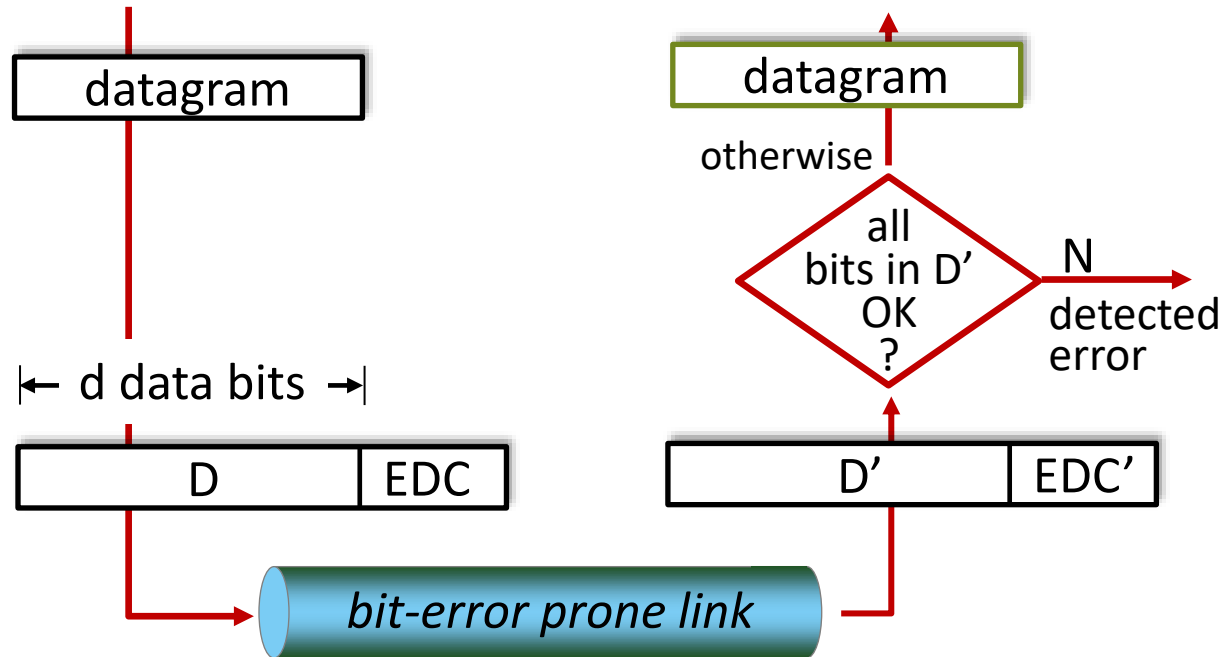
- Looking only to see if any error has occurred
- The answer is a simple yes or no

❑ **Error correction**

- Forward error correction: The process in which the receiver tries to guess the message by using redundant bits
- Retransmission: The receiver detects the occurrence of an error and asks the sender to resend the message

Error Detection

- ❑ EDC: error detection and correction bits (e.g., redundancy)
- ❑ D: data protected by error checking, may include header fields

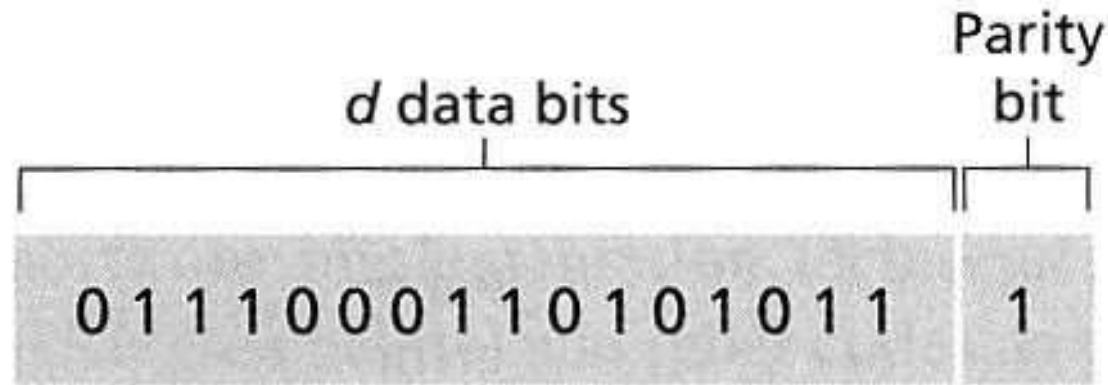


Error detection not 100% reliable!

- protocol may miss some errors, but rarely
- larger EDC field yields better detection and correction

Parity Checking

- Append a parity bit to the end of a block of data (e.g., there are d bits in a block).
- Even parity scheme: the sender includes one additional bit and chooses its value such that the total number of 1s in the $d+1$ bits (the original information plus a parity bit) is even.
- Odd parity scheme: the parity bit value is chosen such that there is an odd number of 1s.



One-bit even parity

Two-dimensional Parity Check

- ❑ Two-dimensional parity is a generalization of single-bit parity.
- ❑ In this scheme, the data is formed as a rectangular matrix j bits wide and i bits high.
- ❑ A parity value is computed for each row and column. It has following properties:
- ❑ A single bit error can be detected.
- ❑ If there is a single error, we can use the column and row indices to identify the bit that was corrupted and correct that error.

Parity Checking



single bit parity:

- detect single bit errors

0111000110101011	1
------------------	---

← d data bits → |
parity bit

Even/odd parity: set parity bit so there is an even/odd number of 1's

At receiver:

- compute parity of d received bits
- compare with received parity bit
– if different then error detected

Can detect *and* correct errors (without retransmission!)

- two-dimensional parity: detect *and correct* **single** bit errors

	row parity →			
	$d_{1,1}$	...	$d_{1,j}$	$d_{1,j+1}$
	$d_{2,1}$	...	$d_{2,j}$	$d_{2,j+1}$
	...	...	...	...
	$d_{i,1}$	...	$d_{i,j}$	$d_{i,j+1}$
column parity ↓	$d_{i+1,1}$	...	$d_{i+1,j}$	$d_{i+1,j+1}$

no errors:

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

detected and correctable single-bit error:

1	0	1	0	1	1
1	0	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

parity error

Cyclic Redundancy Check (CRC)

- CRC treats bit streams as representations of polynomials with coefficients of 0 and 1 only.
 - e.g., 101 can be represented a polynomial as: $1x^2 + 0x^1 + 1x^0 = x^2 + 1$.
- Modulo-2 arithmetic is used for computing CRC.
 - There are no carries for addition or borrows for subtraction.
- When the polynomial code method is employed, the sender and receiver must agree upon a **generator polynomial**, $G(x)$ in advance.

Cyclic Redundancy Check (CRC)



- more powerful error-detection coding
- **D**: data bits (given, think of these as a binary number)
- **G**: bit pattern (generator), of $r+1$ bits (given, specified in CRC standard)



- sender:* compute r CRC bits, **R**, such that $\langle D, R \rangle$ *exactly* divisible by G (**mod 2**)
- receiver knows G , divides $\langle D, R \rangle$ by G . If non-zero remainder: error detected!
 - can detect all burst errors less than $r+1$ bits
 - widely used in practice (Ethernet, 802.11 WiFi)

Cyclic Redundancy Check (CRC) - Example



Sender wants to compute R
such that:

$$D \cdot 2^r \text{ XOR } R = nG$$

... or equivalently (XOR R both sides):

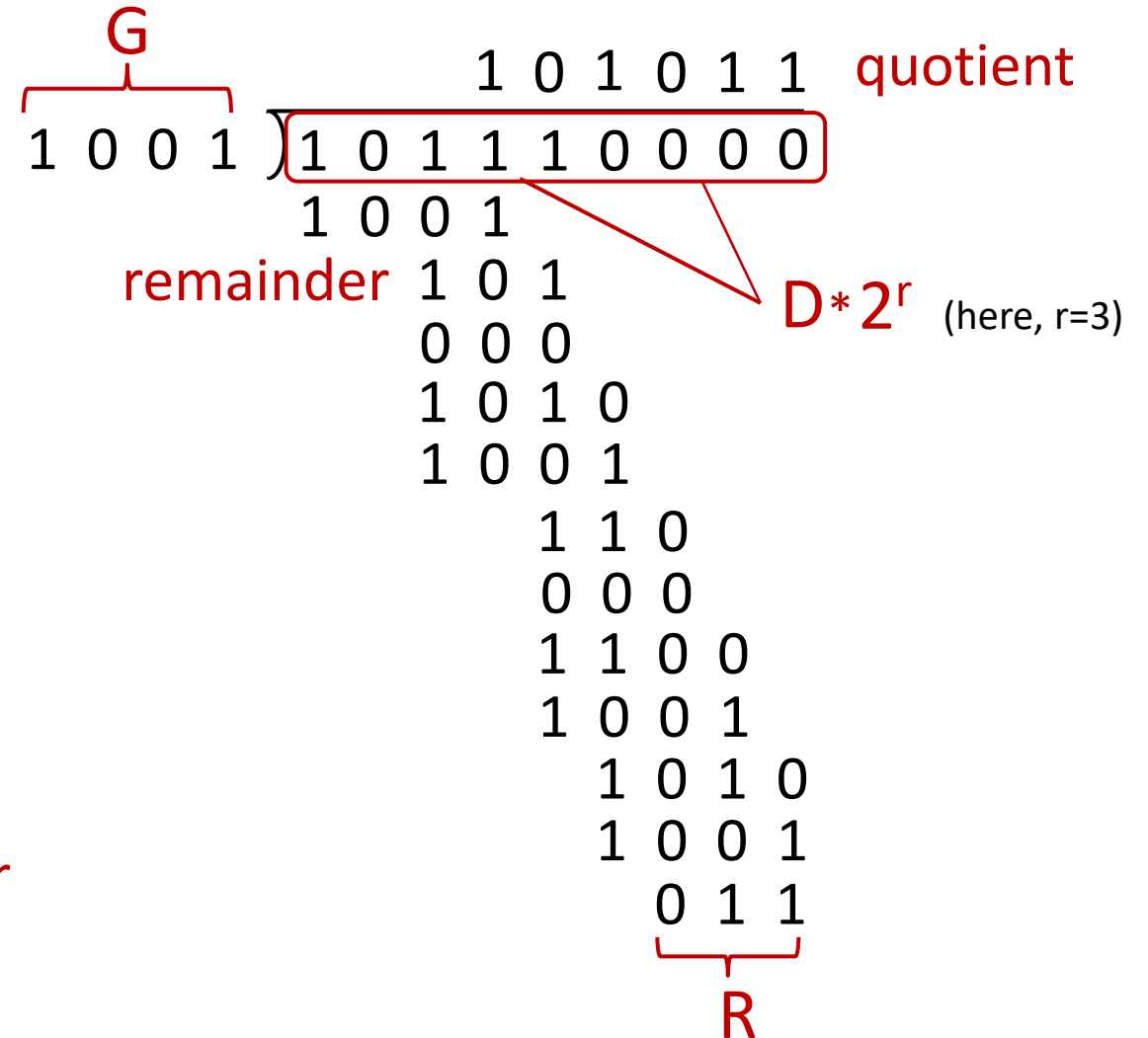
$$D \cdot 2^r = nG \text{ XOR } R$$

... which says:

if we divide $D \cdot 2^r$ by G, we
want remainder R to satisfy:

$$R = \text{remainder} \left[\frac{D \cdot 2^r}{G} \right]$$

*algorithm for
computing R*



Cyclic Redundancy Check - Steps

□ To compute the checksum for some frame with m bits, corresponding to the polynomial $M(x)$, we have following steps:

- Let r be the degree of $G(x)$. Append r zero bits to the low-order end of the frame so it now contains $m+r$ bits and corresponds to the polynomial $x^r M(x)$
 - Divide $G(x)$ into $x^r M(x)$ using modulo-2 division (XOR operation).
 - Subtract the remainder from $x^r M(x)$ using modulo-2 subtraction.
- Append the remainder to the end of $M(x)$ to form the transmitted data frame.

□ To detect the error, the receiver divides the checksummed frame by $G(x)$. If there is a remainder, there has been a transmission error.

CRC – In Class Practice (Not Quiz)

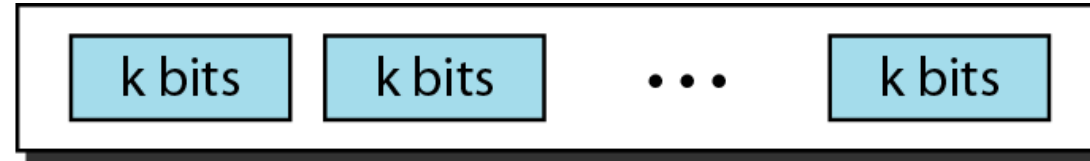
□ You are given the following:

- Data message (M): 1101011011
- Generator polynomial (G): 10011(which represents the divisor)

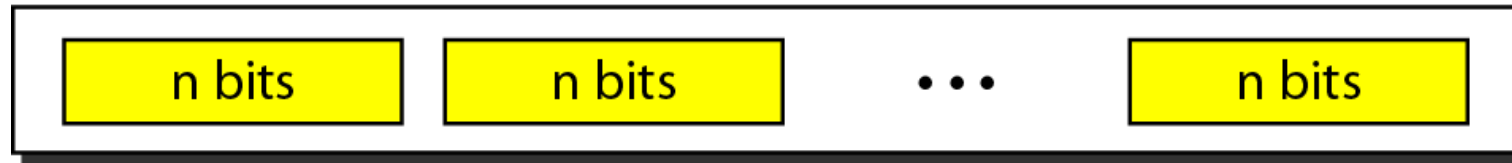
Perform CRC encoding to find the remainder (CRC) when the message is divided by the generator.

Error Correction: Block Coding

The use of error-correcting codes is often referred to as forward error correction (FEC).



2^k Datawords, each of k bits



2^n Codewords, each of n bits (only 2^k of them are valid)

□ In block coding, we divide our message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called **codewords**.

Error Correction: Block Coding

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

A code for error correction when $k = 2$ and $n = 3$

Error Correction: Block Coding

□ Sending dataword “01”

- Receives “011”, Valid
- 1 bit corrupted, receives “111”, Discard
- 2 bit corrupted, receives “000”, decoded as a **wrong** dataword “00”

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Error Correction: Hamming Distance

- The Hamming distance between two codewords is the number of differences between corresponding bits

$10101 \oplus 11110$ is 01011, The Hamming distance is 3 (three 1s)

- The **minimum** Hamming distance is the smallest Hamming distance between all possible pairs in a set of words

$$d(000,011) = 2 \quad d(000,101) = 2 \quad d(000,110) = 2$$

$$d(011,101) = 2 \quad d(011,110) = 2 \quad d(101,110) = 2$$

In this codebook, $d_{min} = 2$

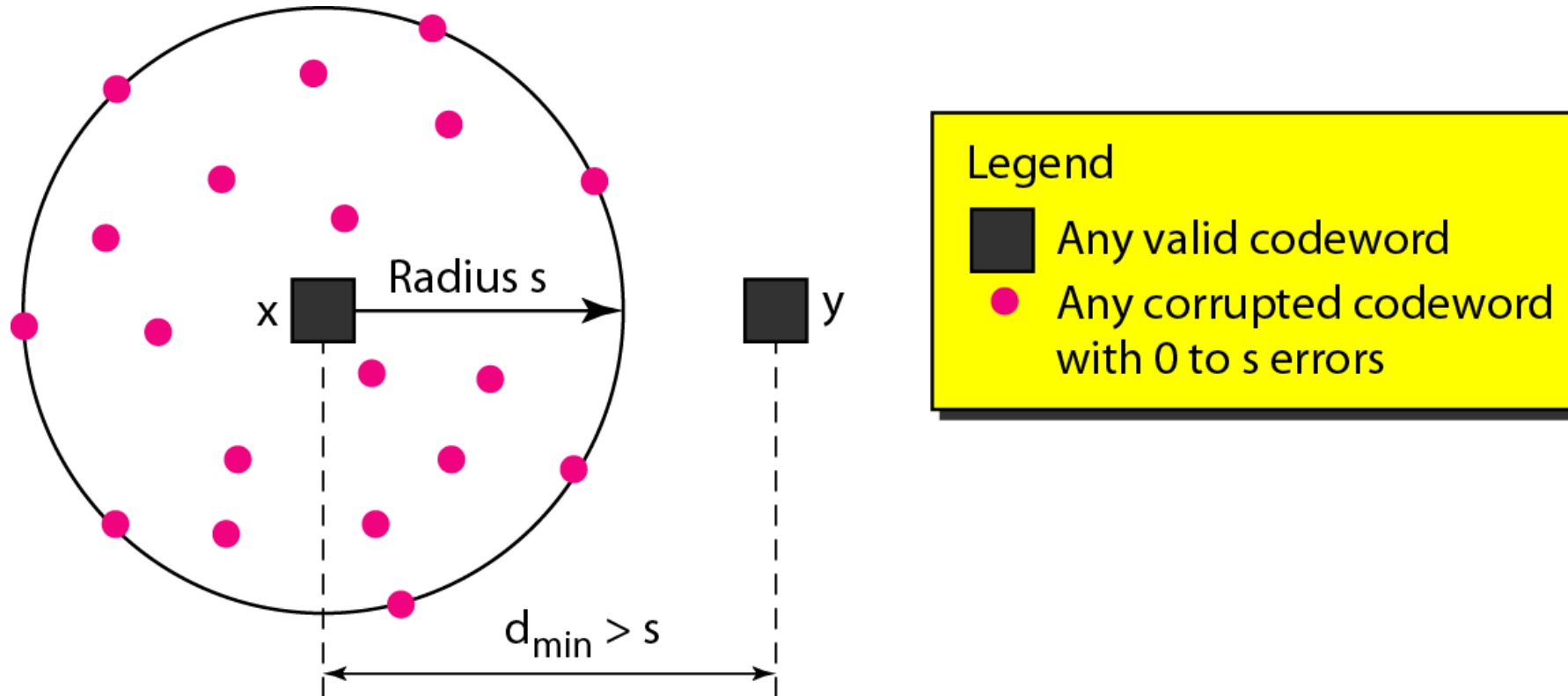
Minimum Hamming Distance

□ To guarantee the **detection** of up to **s bit errors** in all cases, the minimum Hamming distance in a block code must be $d_{min} = s + 1$

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

- $d_{min} = 2$, up to 1 bit error is guaranteed to be detected. **If 1 bit error occurs, the codeword will never change to another codeword, the system will never be fooled to receive a wrong dataword**

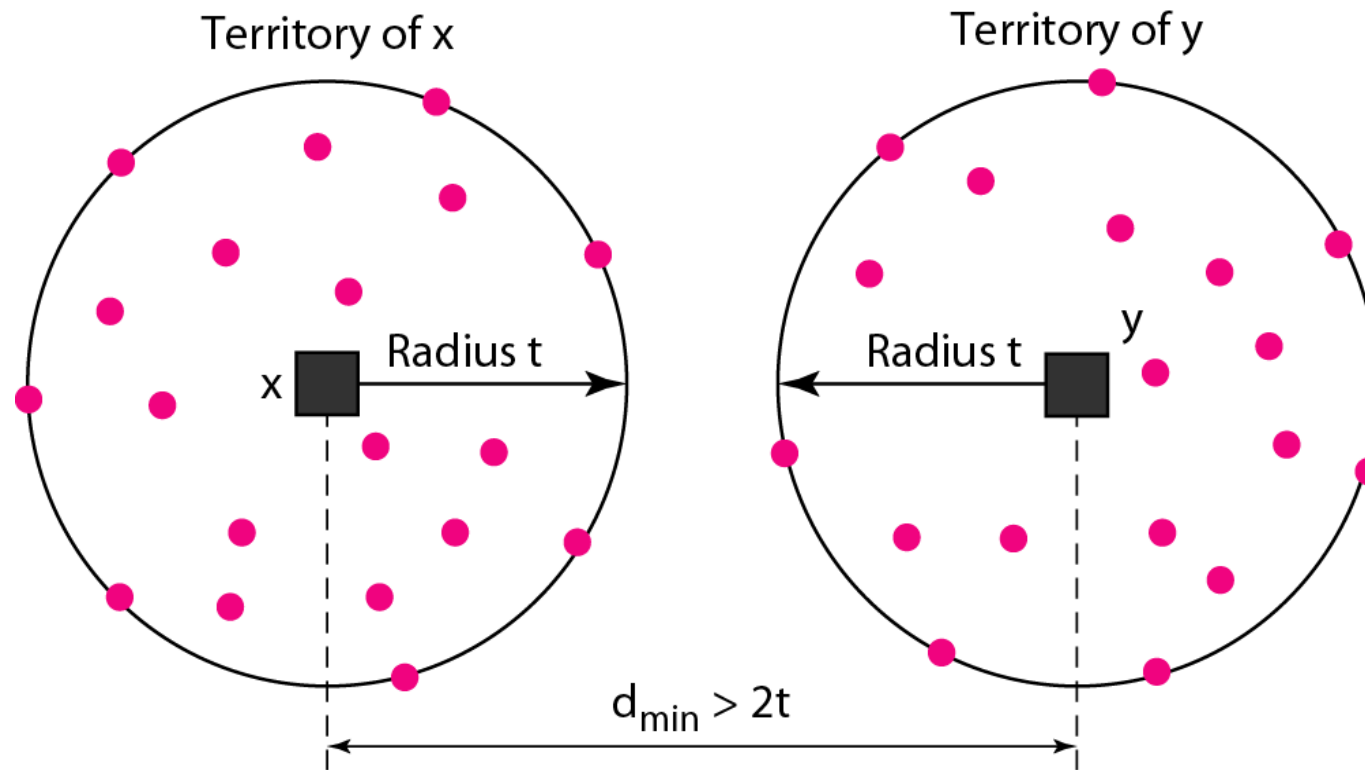
Minimum Hamming Distance



Geometric concept for finding d_{min} in error detection

Minimum Hamming Distance

□ To guarantee the **correction** of up to **t bit errors** in all cases, the minimum Hamming distance in a block code must be **$d_{min} = 2t + 1$**



Legend

- Any valid codeword
- Any corrupted codeword with 1 to t errors

Geometric concept for finding d_{min} in error correction

Hamming Code



- The relationship between number of hamming **check bits k** and the number of **data bits n** is $2^k \geq k + n + 1$

n	1	2-4	5-11	12-26
k	2	3	4	5

Hamming Code - Example

- Assuming data bits $n=4$, the data is “1010”. C_i represents check bit, b_i represents data bit. **Check bit should be placed at each 2^{k-1} position**

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value			1		0	1	0

Question: What is the rule of generating the values of check bits?

Hamming Code - Example

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value			1		0	1	0

$$\begin{array}{rcl}
 H_3 & = & \boxed{0} \quad \boxed{1} \quad \boxed{1} \\
 H_5 & = & \boxed{1} \quad \boxed{0} \quad \boxed{1} \\
 H_6 & = & \boxed{1} \quad \boxed{1} \quad \boxed{0} \\
 H_7 & = & \boxed{1} \quad \boxed{1} \quad \boxed{1} \\
 & & C_4 \quad C_2 \quad C_1
 \end{array}$$

C_1 is responsible for the position where **last bit equal to 1**

C_2 is responsible for the position where **second last bit equal to 1**

C_3 is responsible for the position where **first bit equal to 1**

Hamming Code - Example

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value			1		0	1	0

$$\begin{array}{rcl}
 H_3 & = & \boxed{0} \quad \boxed{1} \quad \boxed{1} \\
 H_5 & = & \boxed{1} \quad \boxed{0} \quad \boxed{1} \\
 H_6 & = & \boxed{1} \quad \boxed{1} \quad \boxed{0} \\
 H_7 & = & \boxed{1} \quad \boxed{1} \quad \boxed{1} \\
 & & C_4 \quad C_2 \quad C_1
 \end{array}$$

$$C_1 = H_3 \oplus H_5 \oplus H_7 = b_1 \oplus b_2 \oplus b_4 = 1 \oplus 0 \oplus 0 = 1$$

$$C_2 = H_3 \oplus H_6 \oplus H_7 = b_1 \oplus b_3 \oplus b_4 = 1 \oplus 1 \oplus 0 = 0$$

$$C_4 = H_5 \oplus H_6 \oplus H_7 = b_2 \oplus b_3 \oplus b_4 = 0 \oplus 1 \oplus 0 = 1$$

Hamming Code – Correction Process

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value	1	0	1	1	0	1	0

$$\begin{array}{rcl}
 H_3 & = & \boxed{0} \quad \boxed{1} \quad \boxed{1} \\
 H_5 & = & \boxed{1} \quad \boxed{0} \quad \boxed{1} \\
 H_6 & = & \boxed{1} \quad \boxed{1} \quad \boxed{0} \\
 H_7 & = & \boxed{1} \quad \boxed{1} \quad \boxed{1} \\
 & & C_4 \quad C_2 \quad C_1
 \end{array}$$

$$P_1 = C_1 \oplus b_1 \oplus b_2 \oplus b_4 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$P_2 = C_2 \oplus b_1 \oplus b_3 \oplus b_4 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$P_4 = C_4 \oplus b_2 \oplus b_3 \oplus b_4 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$P_4 P_2 P_1 = 000, \text{ all bits are correct}$$

Hamming Code – Correction Process

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value	1	0	1	1	0	1	0 -> 1

An error!

$$\begin{array}{rcl}
 H_3 & = & \boxed{0} \quad \boxed{1} \quad \boxed{1} \\
 H_5 & = & \boxed{1} \quad \boxed{0} \quad \boxed{1} \\
 H_6 & = & \boxed{1} \quad \boxed{1} \quad \boxed{0} \\
 H_7 & = & \boxed{1} \quad \boxed{1} \quad \boxed{1} \\
 & & C_4 \quad C_2 \quad C_1
 \end{array}$$

$$P_1 = C_1 \oplus b_1 \oplus b_2 \oplus b_4 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$P_2 = C_2 \oplus b_1 \oplus b_3 \oplus b_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$P_4 = C_4 \oplus b_2 \oplus b_3 \oplus b_4 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$P_4 P_2 P_1 = 111$, the 7th bit is wrong

Hamming Code – Correction Process

Binary Pos	H_1	H_2	H_3	H_4	H_5	H_6	H_7
Entity	C_1	C_2	b_1	C_4	b_2	b_3	b_4
Value	1	0	1	1	0	1 -> 0	0

An error!

$$\begin{array}{rcl}
 H_3 & = & \boxed{0} \quad \boxed{1} \quad \boxed{1} \\
 H_5 & = & \boxed{1} \quad \boxed{0} \quad \boxed{1} \\
 H_6 & = & \boxed{1} \quad \boxed{1} \quad \boxed{0} \\
 H_7 & = & \boxed{1} \quad \boxed{1} \quad \boxed{1} \\
 & & C_4 \quad C_2 \quad C_1
 \end{array}$$

$$P_1 = C_1 \oplus b_1 \oplus b_2 \oplus b_4 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

$$P_2 = C_2 \oplus b_1 \oplus b_3 \oplus b_4 = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$P_4 = C_4 \oplus b_2 \oplus b_3 \oplus b_4 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$P_4 P_2 P_1 = 110$, the 6th bit is wrong

Hamming Code – In Class Practice (Not Quiz)

- Compute the hamming code (including check bits and data bits), with data bits = “1001000”

Flow Control

- ❑ **To control the transmission rate between a sender and a receiver, two approaches are used.**
 - Feedback-based flow control: the receiver sends back its status to the sender.
 - Rate-based flow control: a built-in mechanism is used to limit the rate at which senders may transmit data.

Taxonomy of Flow Control protocols

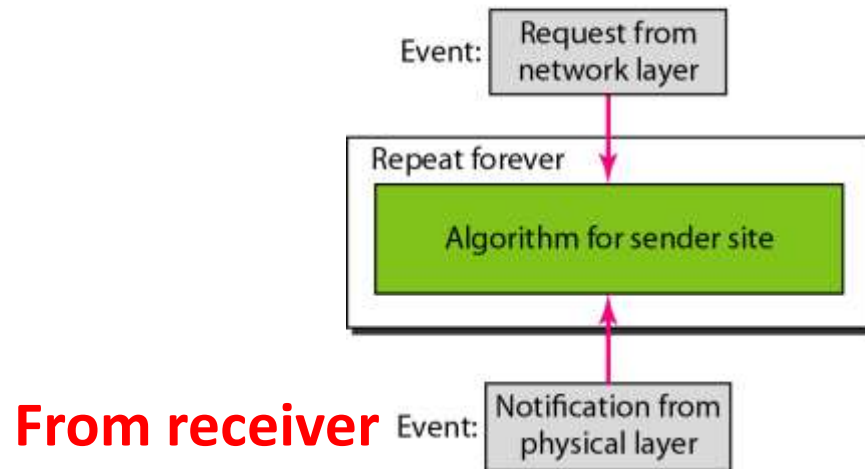
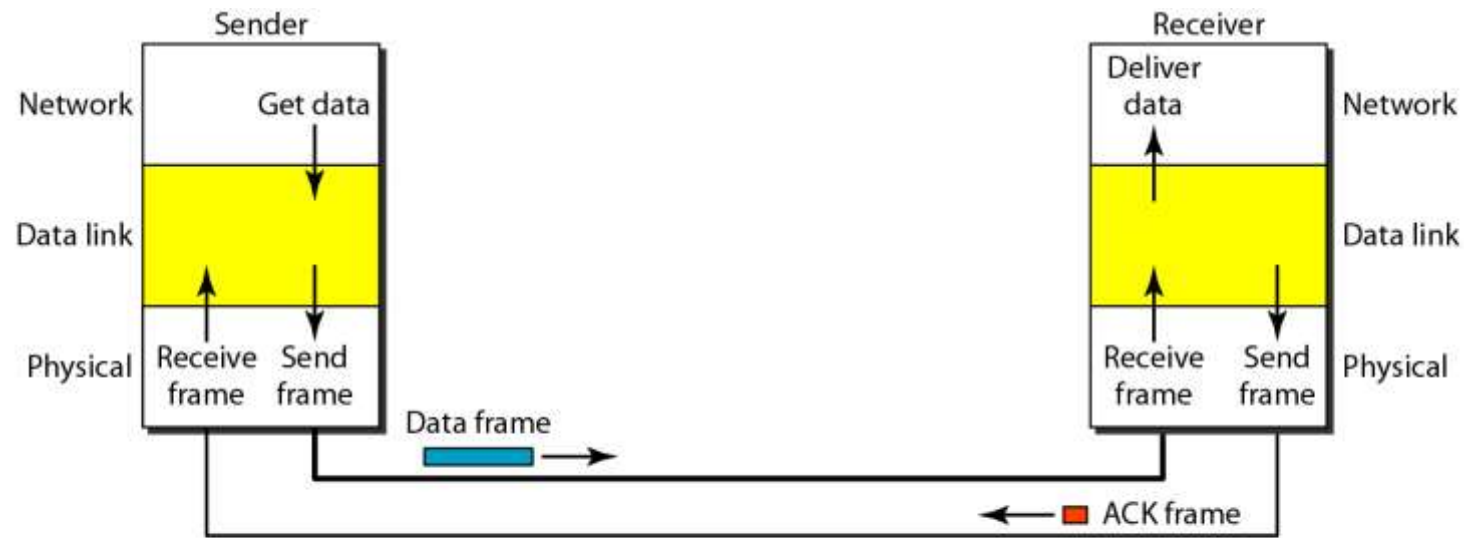
□ Stop-and-Wait Protocol

- The sender sends one frame and then waits for an acknowledgement before proceeding

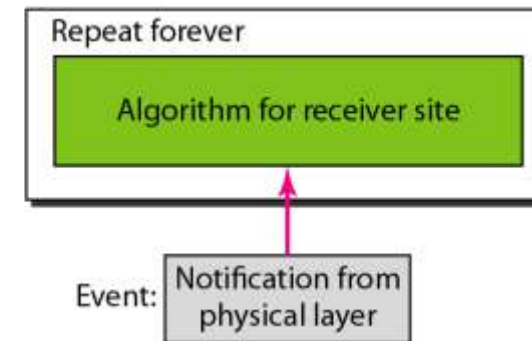
□ Sliding Window Protocols

- One-Bit Sliding Window Protocol, A sliding window protocol with a maximum window size of 1 (Stop-and-Wait protocol).
- Go-back-N Protocol
- Selective Repeat

Stop-and-Wait Protocol

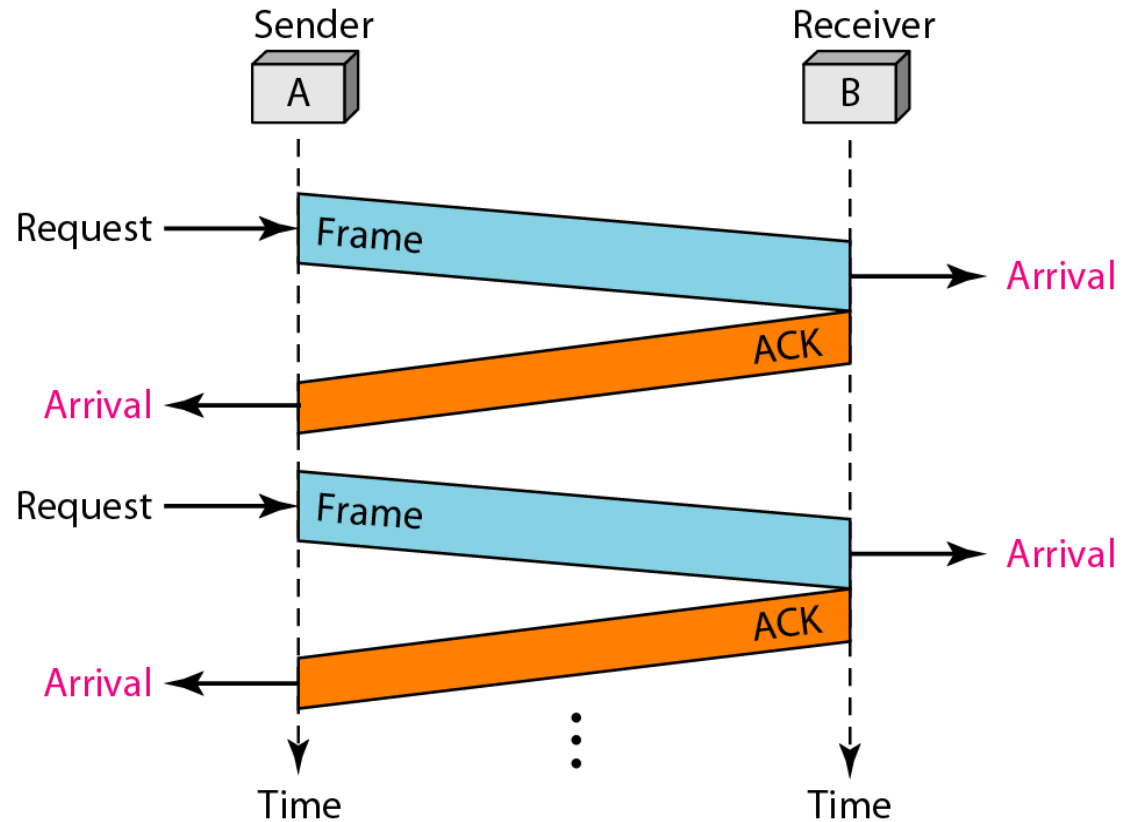


ACK frame from receiver



Flow of Stop-and-Wait Protocol

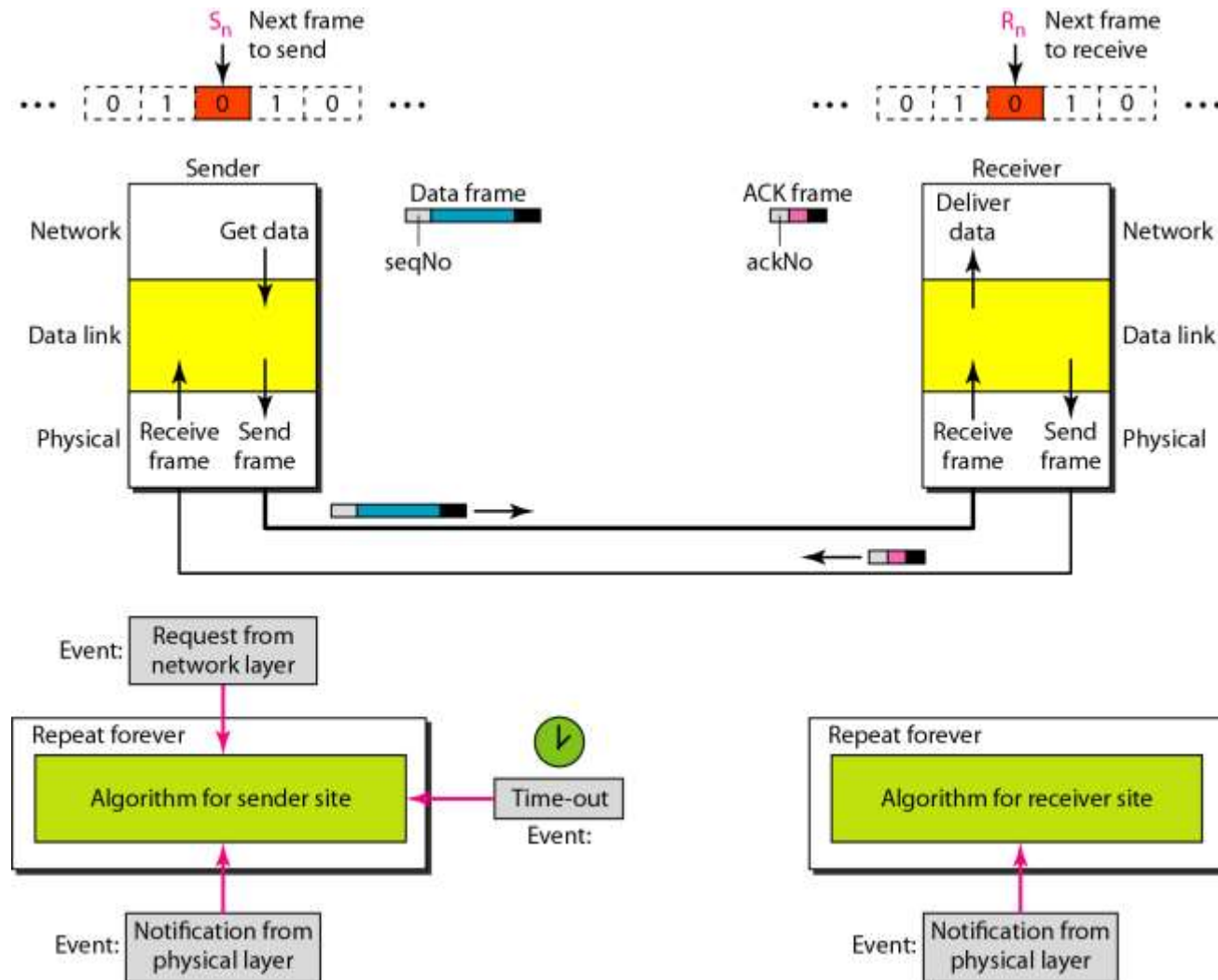
- The sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame. Note that sending two frames in the protocol involves the sender in four events and the receiver in two events.



Stop-and-Wait ARQ Protocol

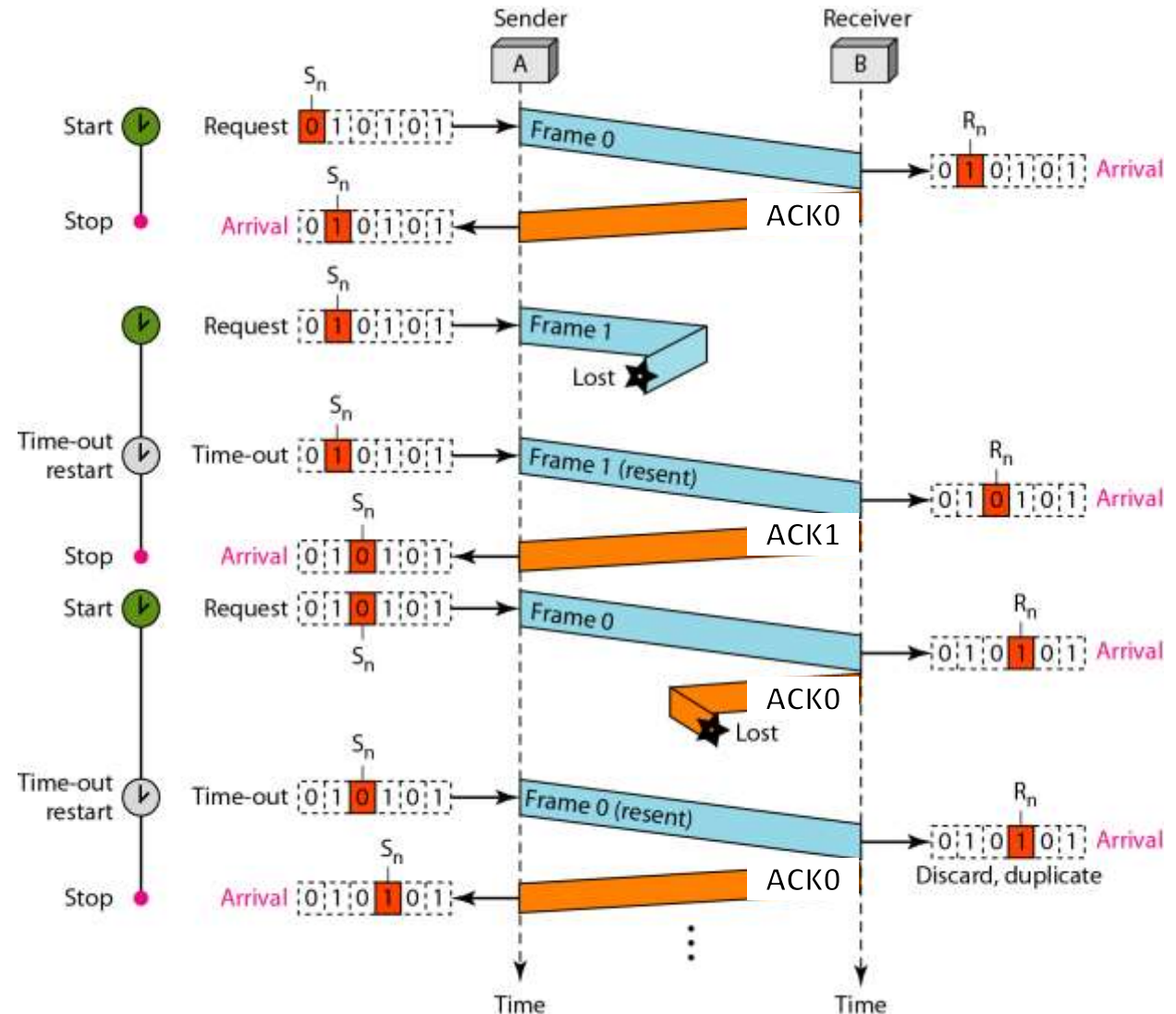
- ❑ Error correction in Stop-and-Wait ARQ (Automatic Repeat reQuest) is done by keeping a copy of the sent frame and retransmitting of the frame when the timer expires
- ❑ In Stop-and-Wait ARQ, we use sequence numbers to number the frames. The sequence numbers are based on modulo-2 arithmetic
- ❑ In Stop-and-Wait ARQ, the acknowledgment number always announces in modulo-2 arithmetic the sequence number of the next frame expected

Design of the Stop-and-Wait ARQ Protocol



Flow of Stop-and-Wait ARQ Protocol

□ Frame 0 is sent and acknowledged. Frame 1 is lost and resent after the time-out. The resent frame 1 is acknowledged and the timer stops. Frame 0 is sent and acknowledged, but the acknowledgment is lost. The sender has no idea if the frame or the acknowledgment is lost, so after the time-out, it resends frame 0, which is acknowledged



Sliding Window Protocols

❑ One-Bit Sliding Window Protocol

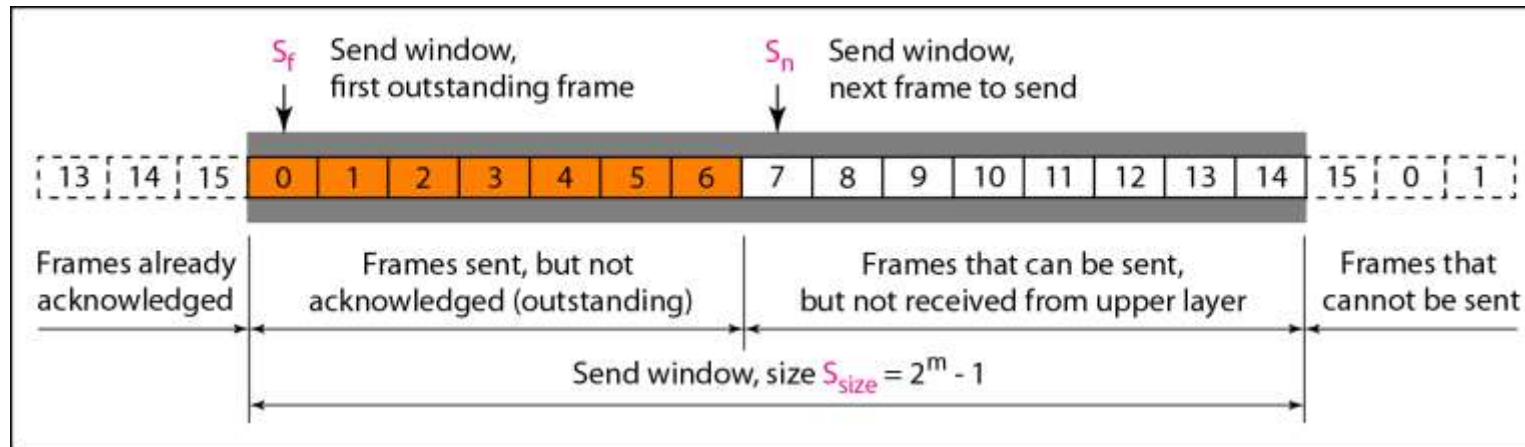
- A sliding window protocol with a maximum window size of 1 (**stop- and-wait protocol**).
- What is the disadvantage for the One-Bit Sliding Window?

❑ Go-back-N Protocol

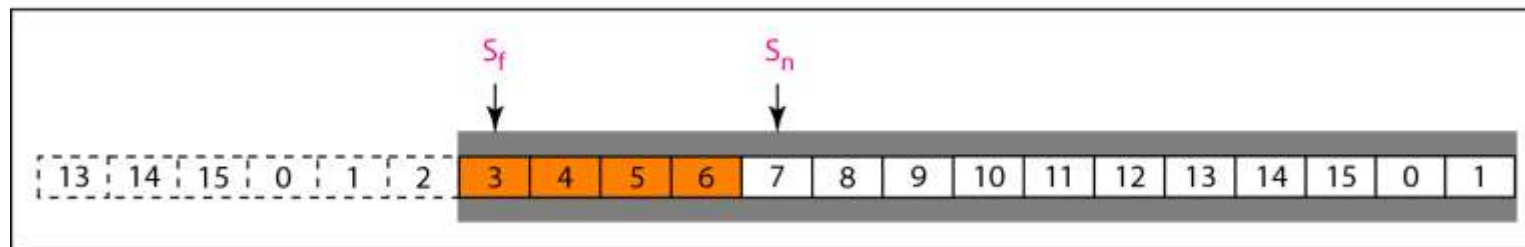
- A sliding window protocol with a maximum windows size of w (where w is larger than 1).
- It is a need for a large window on the sending side occurs whenever the product of bandwidth & round-trip-delay is large. *If the bandwidth is high, the sender will exhaust its window quickly. If the delay is high, the sender will exhaust its window quickly.*
- *Two approaches are used to deal with errors during transmission: go-back-n and selective repeat*

Go-Back-N ARQ Protocol

- ❑ In the Go-Back-N Protocol, the sequence numbers are modulo 2^m , where m is the size of the sequence number field in bits



a. Send window before sliding



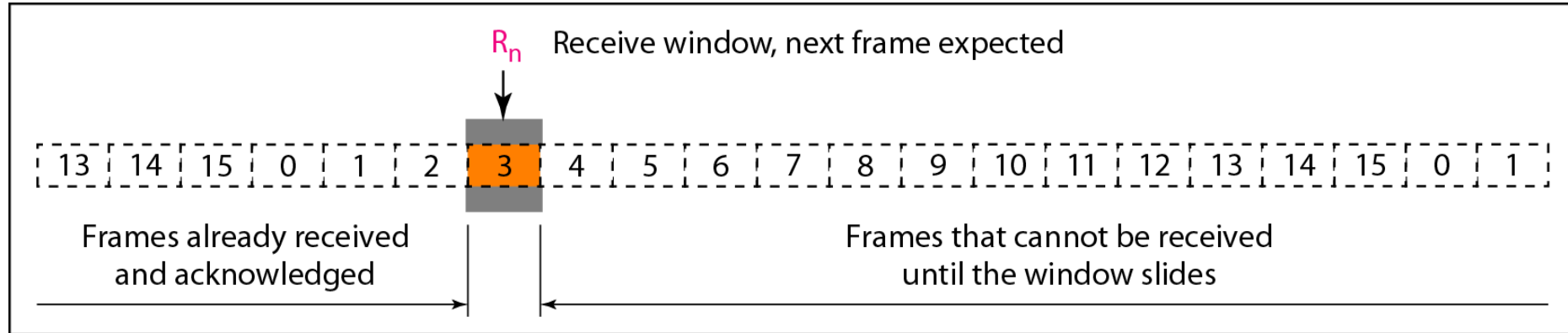
b. Send window after sliding

Send window for Go-Back-N ARQ

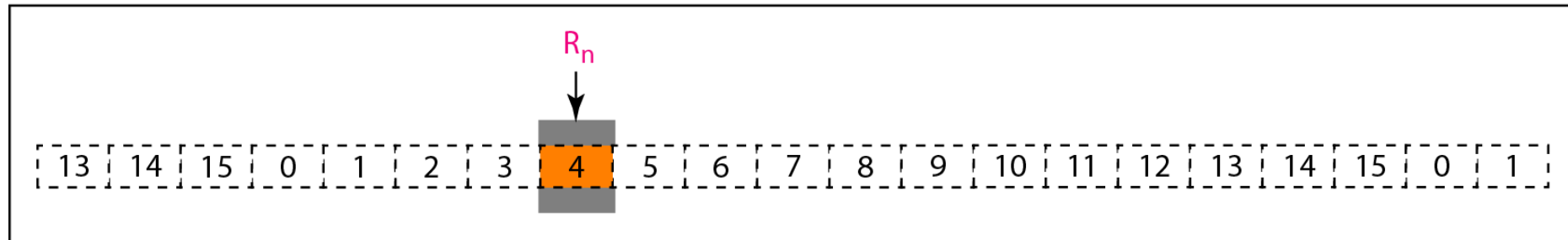
Go-Back-N ARQ Protocol

- The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: S-f, S-n, and S-size.
- The send window can slide one or more slots when a valid acknowledgment arrives
- **Stop-and-Wait ARQ is a special case of Go-Back-N ARQ in which the size of the send window is 1.**

Receive window for Go-Back-N ARQ



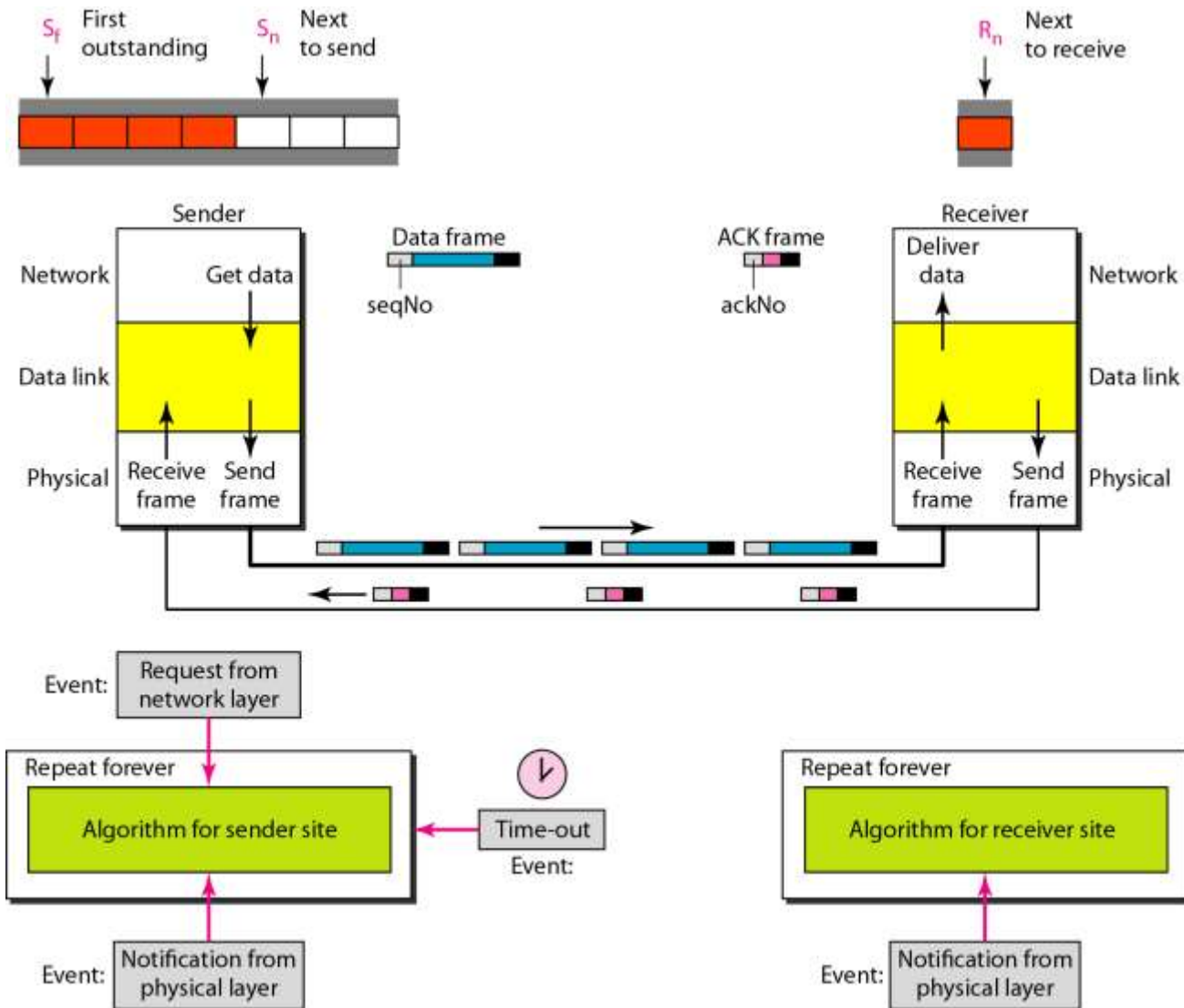
a. Receive window



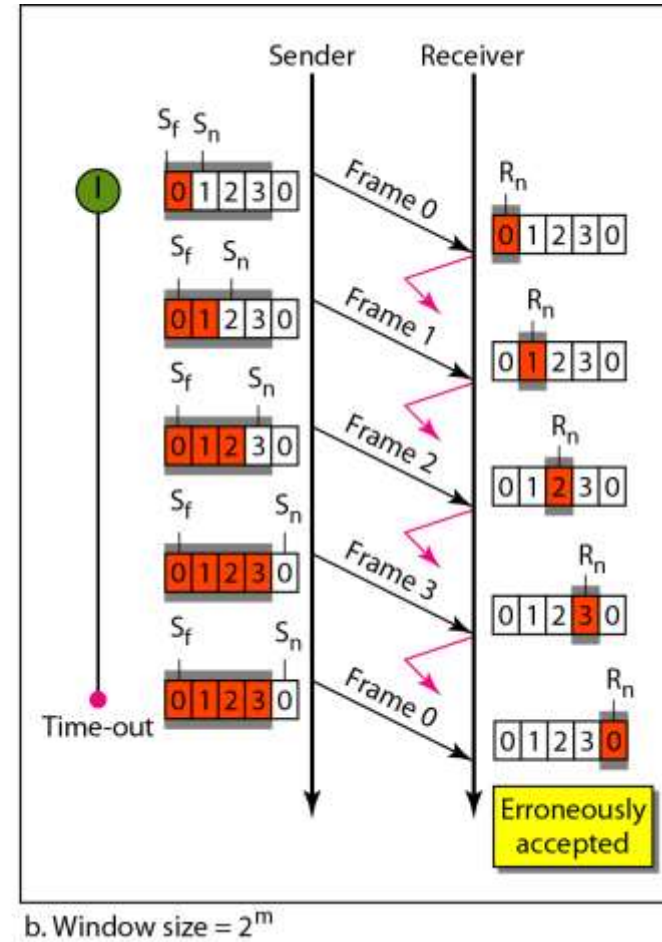
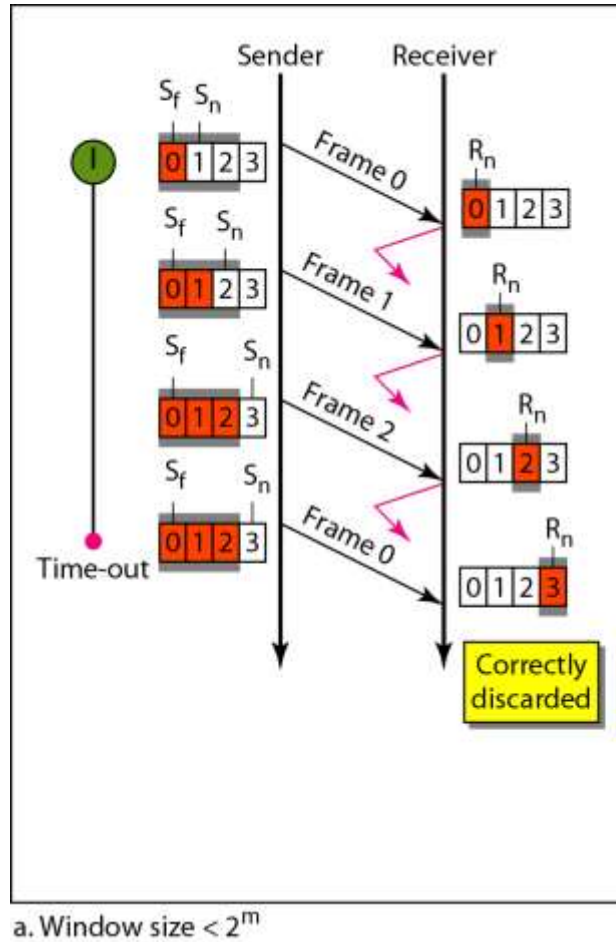
b. Window after sliding

The receive window is an abstract concept defining an imaginary box of size 1 with one single variable R_n . The window slides when a correct frame has arrived; sliding occurs one slot at a time

Design of Go-Back-N ARQ



Window size for Go-Back-N ARQ

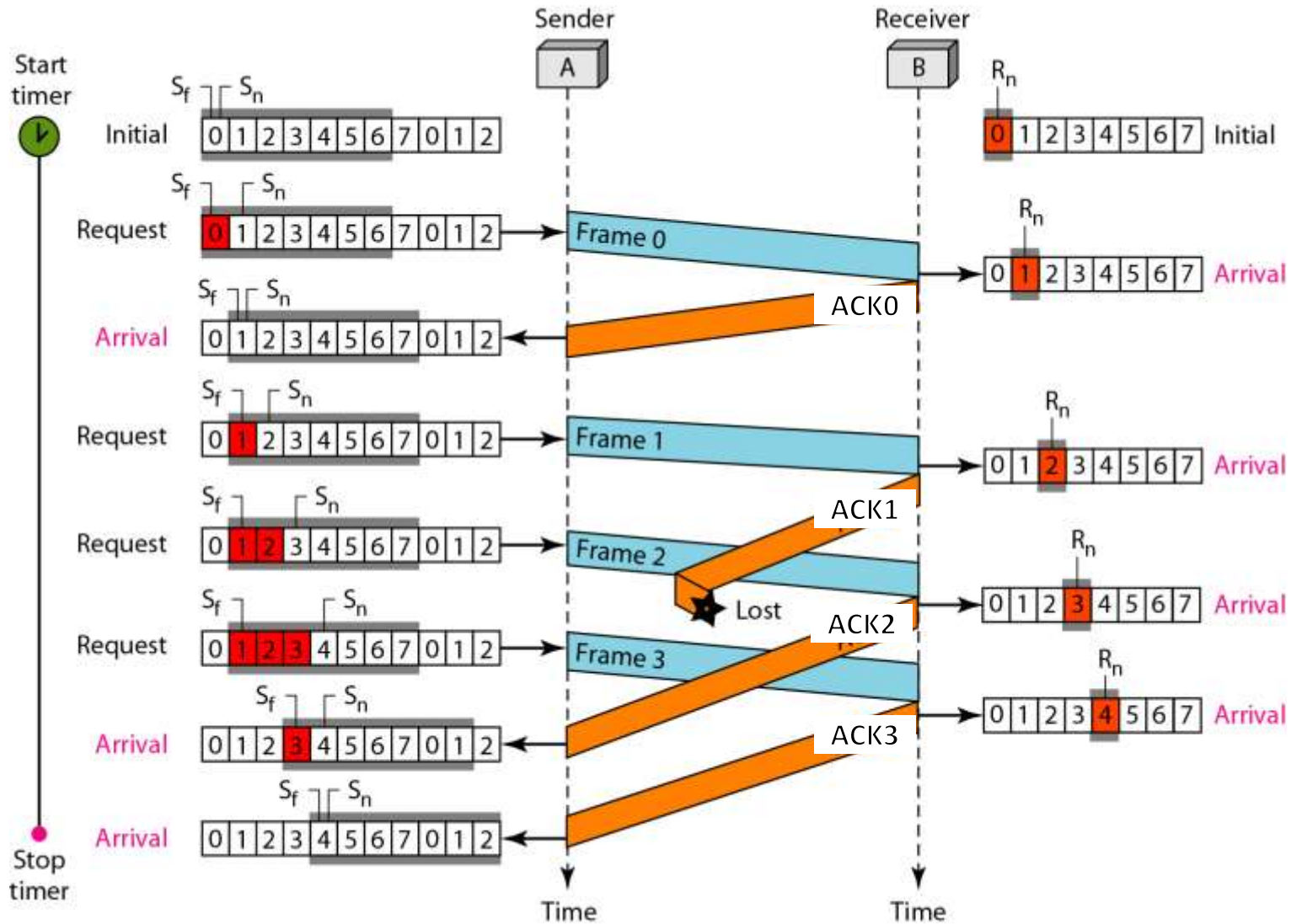


In Go-Back-N ARQ, the size of the send window must be less than 2^m ; the size of the receiver window is always 1

Example 4.1

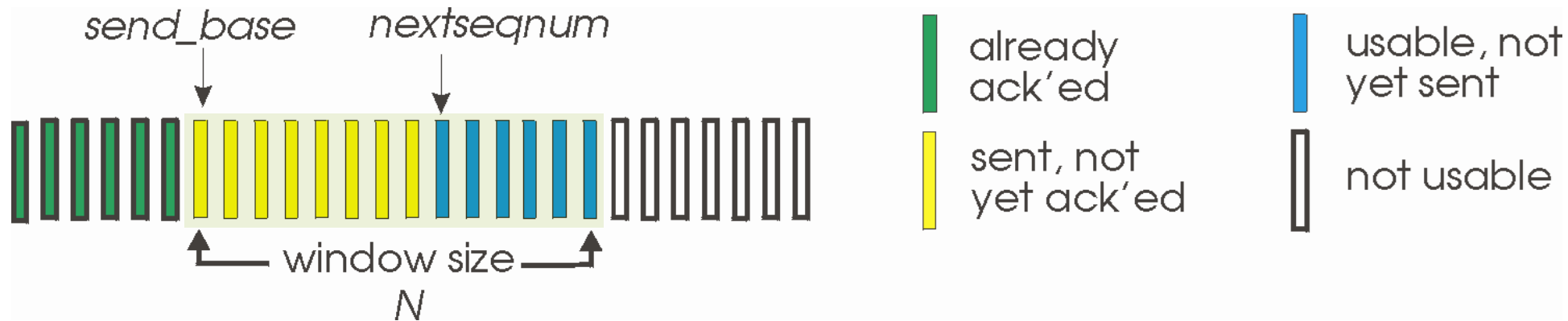
Figure 4.1 shows an example of Go-Back-N. This is an example of a case where the forward channel is reliable, but the reverse is not. No data frames are lost, but some ACKs are delayed and one is lost. The example also shows how cumulative acknowledgments can help if acknowledgments are delayed or lost. After initialization, there are seven sender events. Request events are triggered by data from the network layer; arrival events are triggered by acknowledgments from the physical layer. There is no time-out event here because all outstanding frames are acknowledged before the timer expires. Note that although ACK 1 is lost, ACK 2 serves as both ACK 1 and ACK 2.

Figure 4.1



Go-Back-N: sender

- sender: “window” of up to N , consecutive transmitted but unACKed pkts
 - k -bit seq # in pkt header

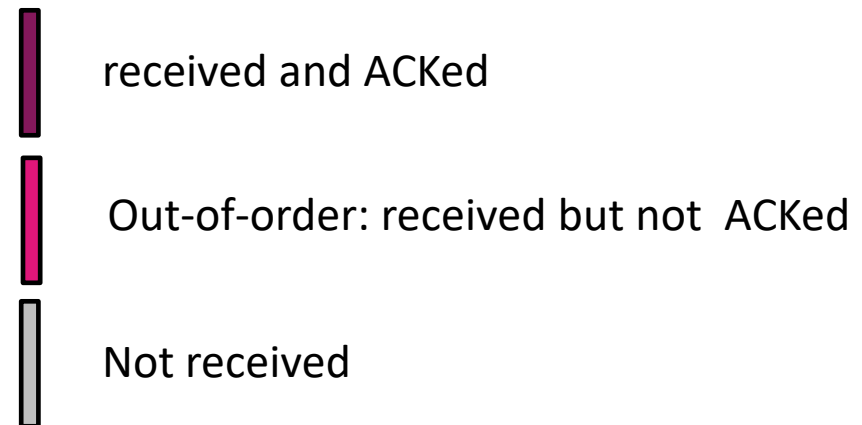
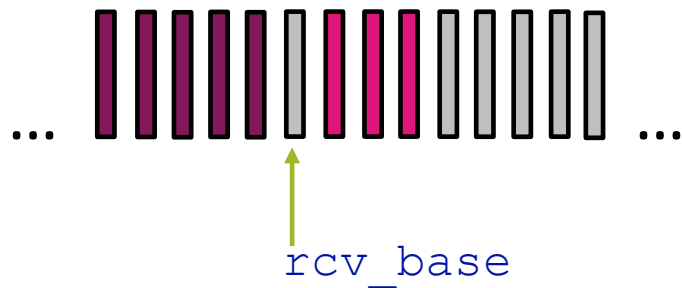


- ***cumulative ACK***: $ACK(n)$: ACKs all packets up to, including seq # n
 - on receiving $ACK(n)$: move window forward to begin at $n+1$
- timer for oldest in-flight packet
- ***timeout(n)***: retransmit packet n and all higher seq # packets in window

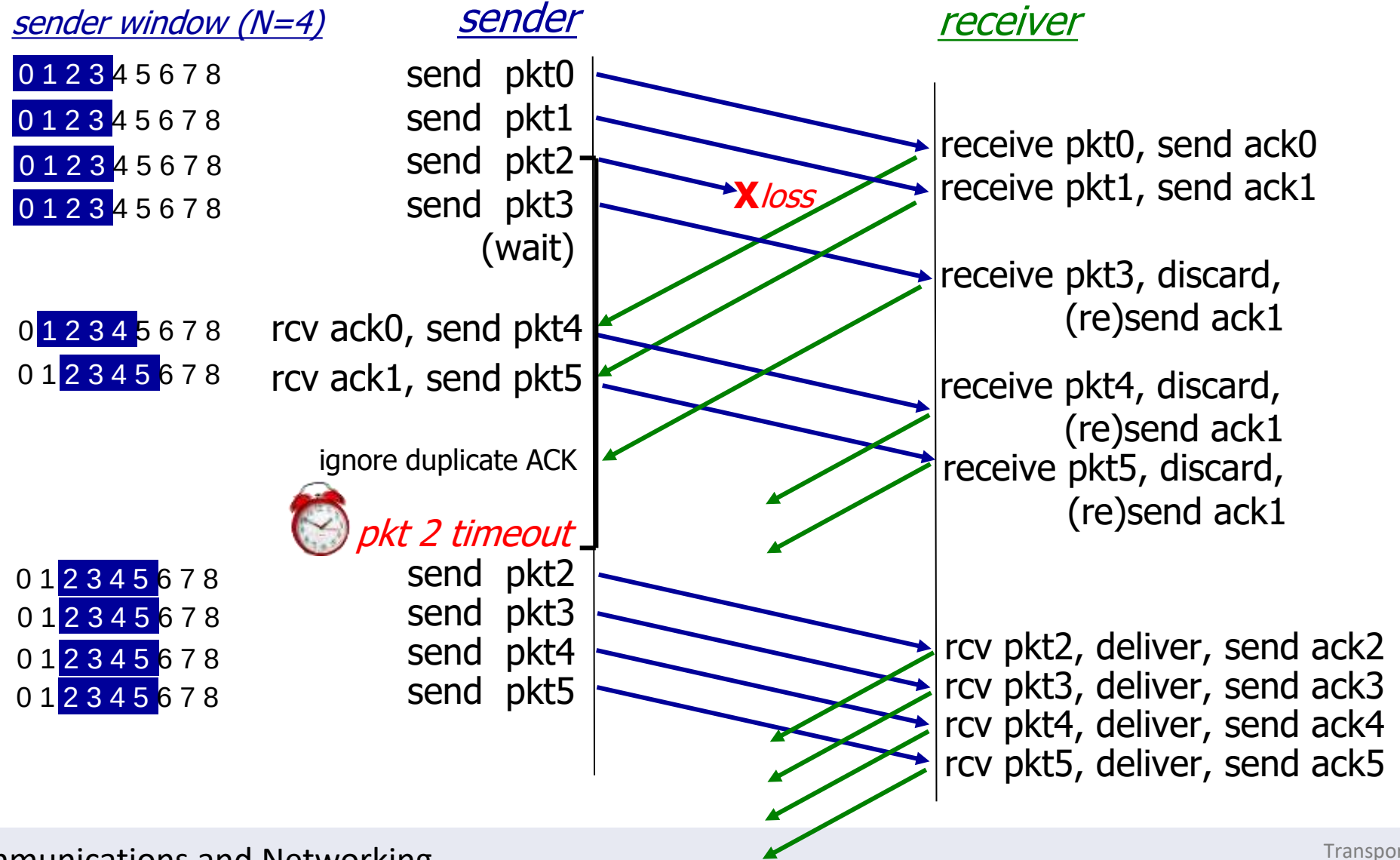
Go-Back-N: receiver

- ACK-only: always send ACK for correctly-received packet so far, with highest *in-order* seq #
 - may generate duplicate ACKs
 - need only remember `rcv_base`
- on receipt of out-of-order packet:
 - can discard (don't buffer) or buffer: an implementation decision
 - re-ACK pkt with highest in-order seq #

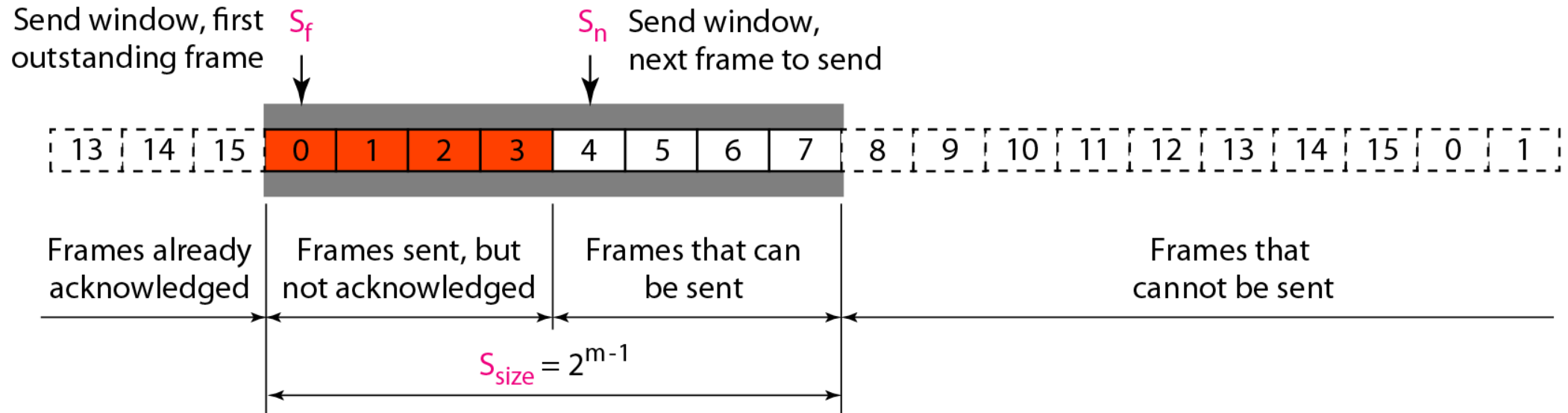
Receiver view of sequence number space:



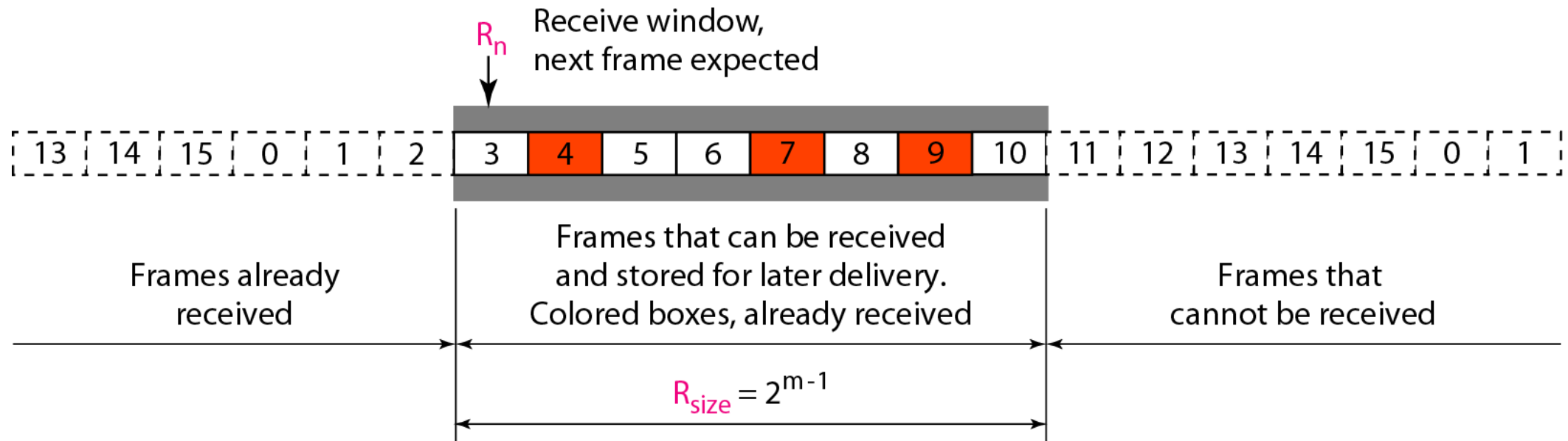
Go-Back-N in action



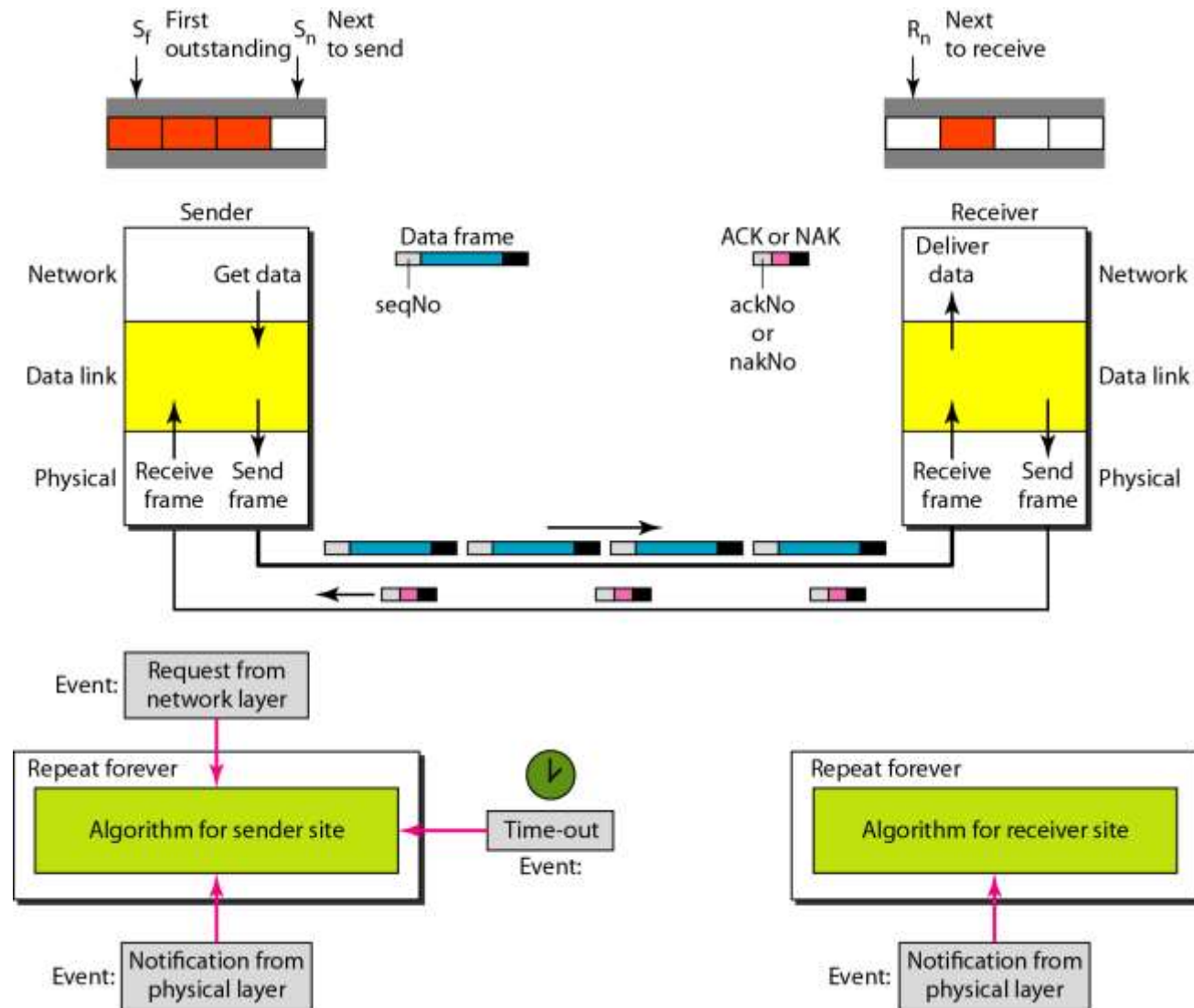
Send window for Selective Repeat ARQ



Receive window for Selective Repeat ARQ



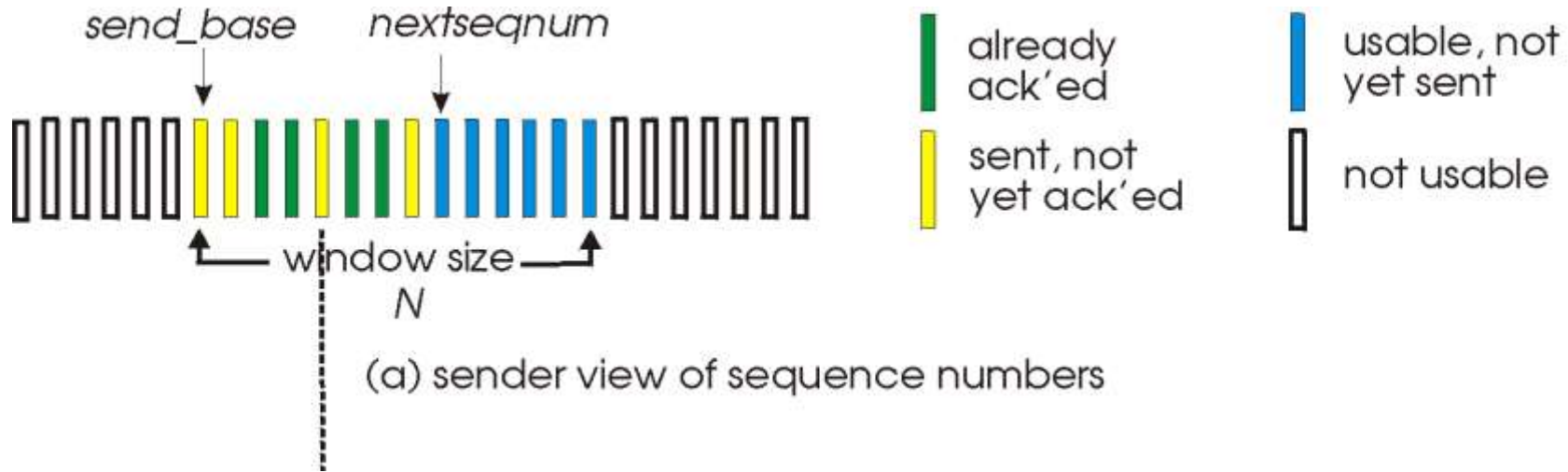
Design of Selective Repeat ARQ



COMP3003: Data Communications and Networking



Selective repeat: sender, receiver windows



Selective repeat: sender and receiver

sender

data from above:

- if next available seq # in window, send packet

timeout(n):

- resend packet n , restart timer

ACK(n) in [sendbase, sendbase+N-1]:

- mark packet n as received
- if n smallest unACKed packet, advance window base to next unACKed seq #

receiver

packet n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order packets), advance window to next not-yet-received packet

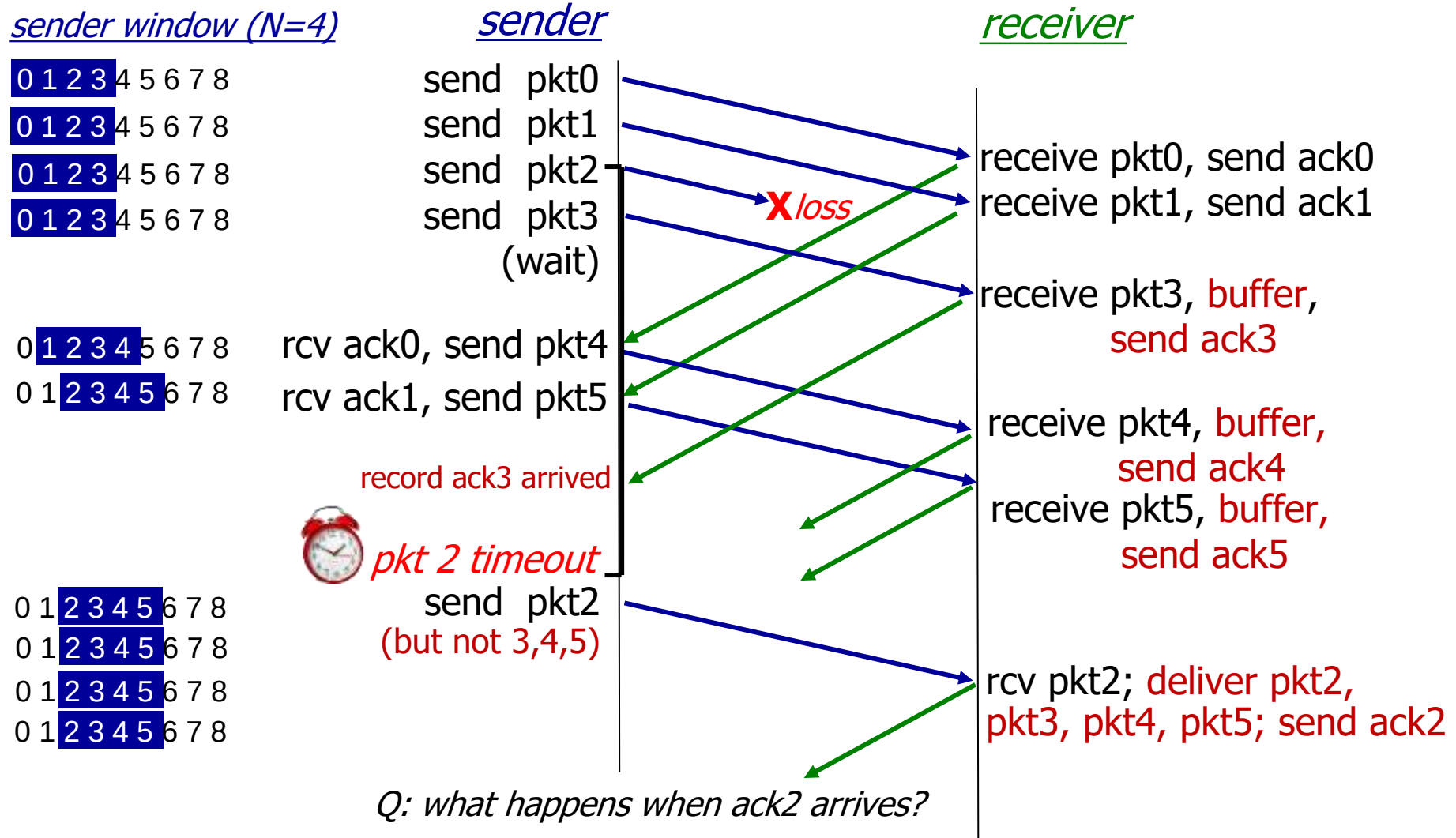
packet n in [rcvbase-N, rcvbase-1]

- ACK(n)

otherwise:

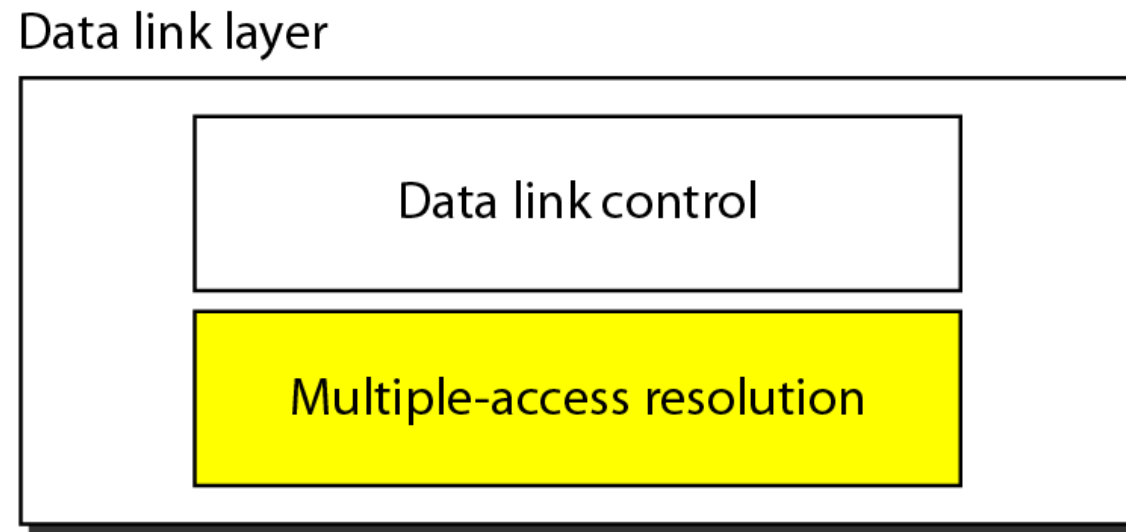
- ignore

Selective Repeat in action

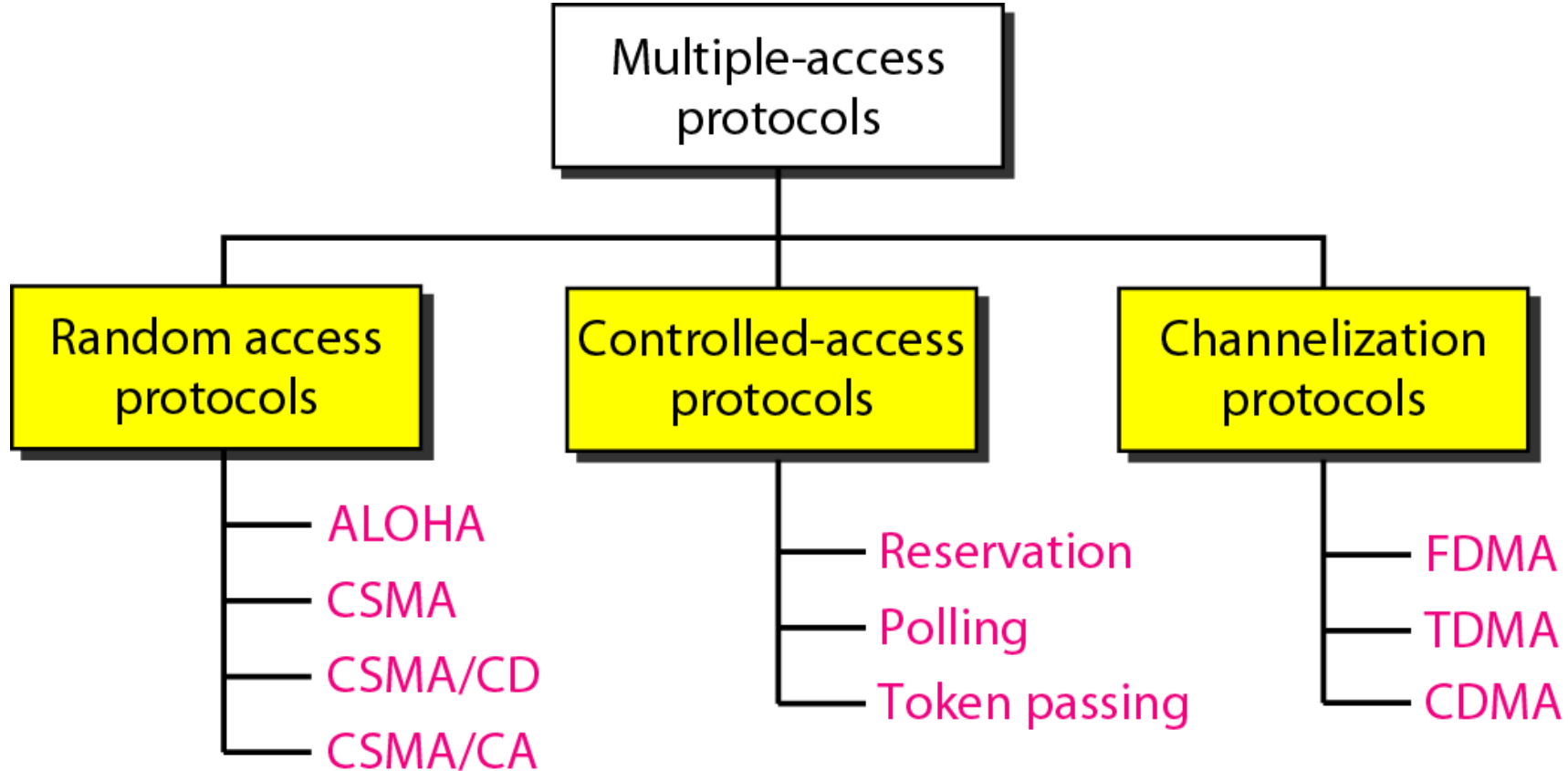


Multi-Access

- Data link layer divided into two functionality-oriented sublayers



Taxonomy of multiple-access protocols

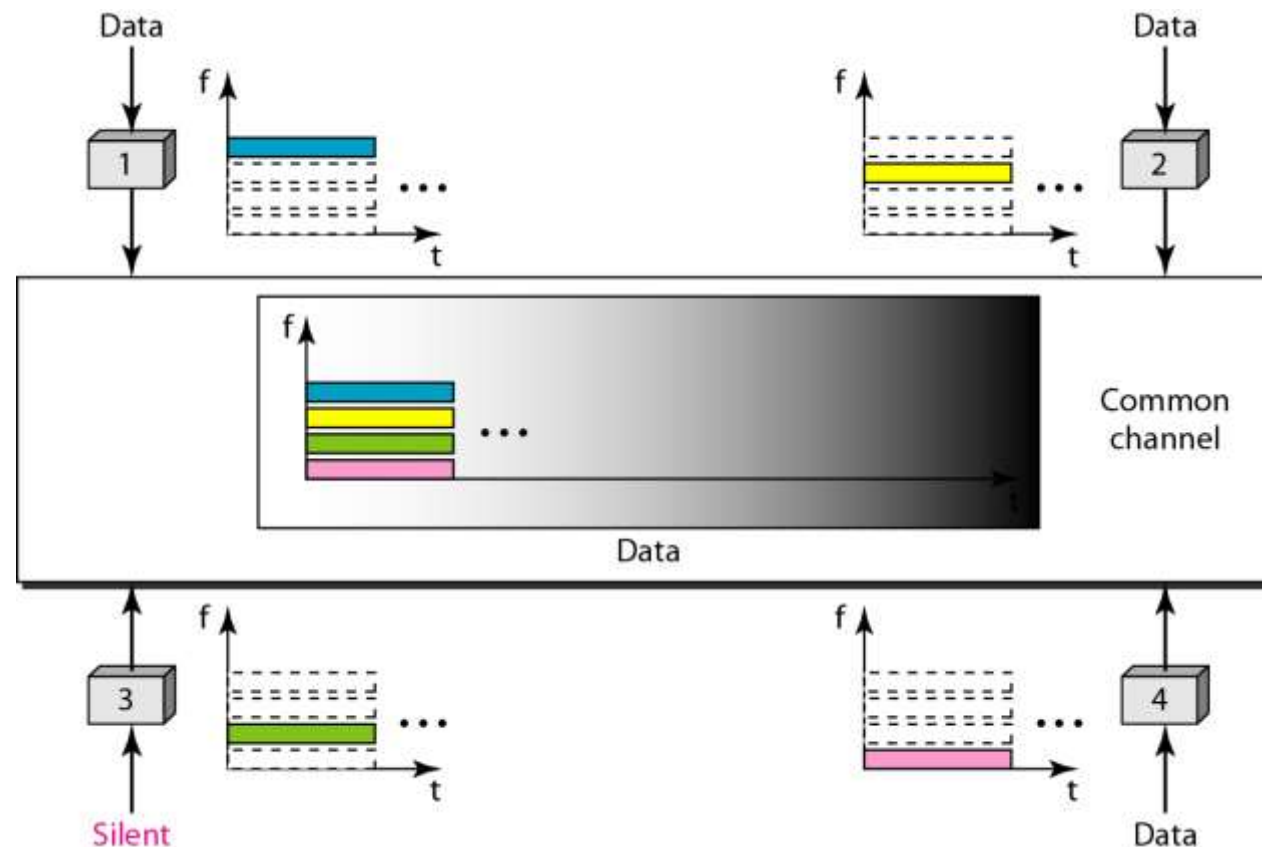


Channelization

- ❑ Channelization is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, between different stations. In this section, we discuss three channelization protocols
 - Frequency-Division Multiple Access (FDMA)
 - Time-Division Multiple Access (TDMA)
 - Code-Division Multiple Access (CDMA)

Frequency-division multiple access (FDMA) ★

- In FDMA, the available bandwidth of the common channel is divided into bands that are separated by guard bands

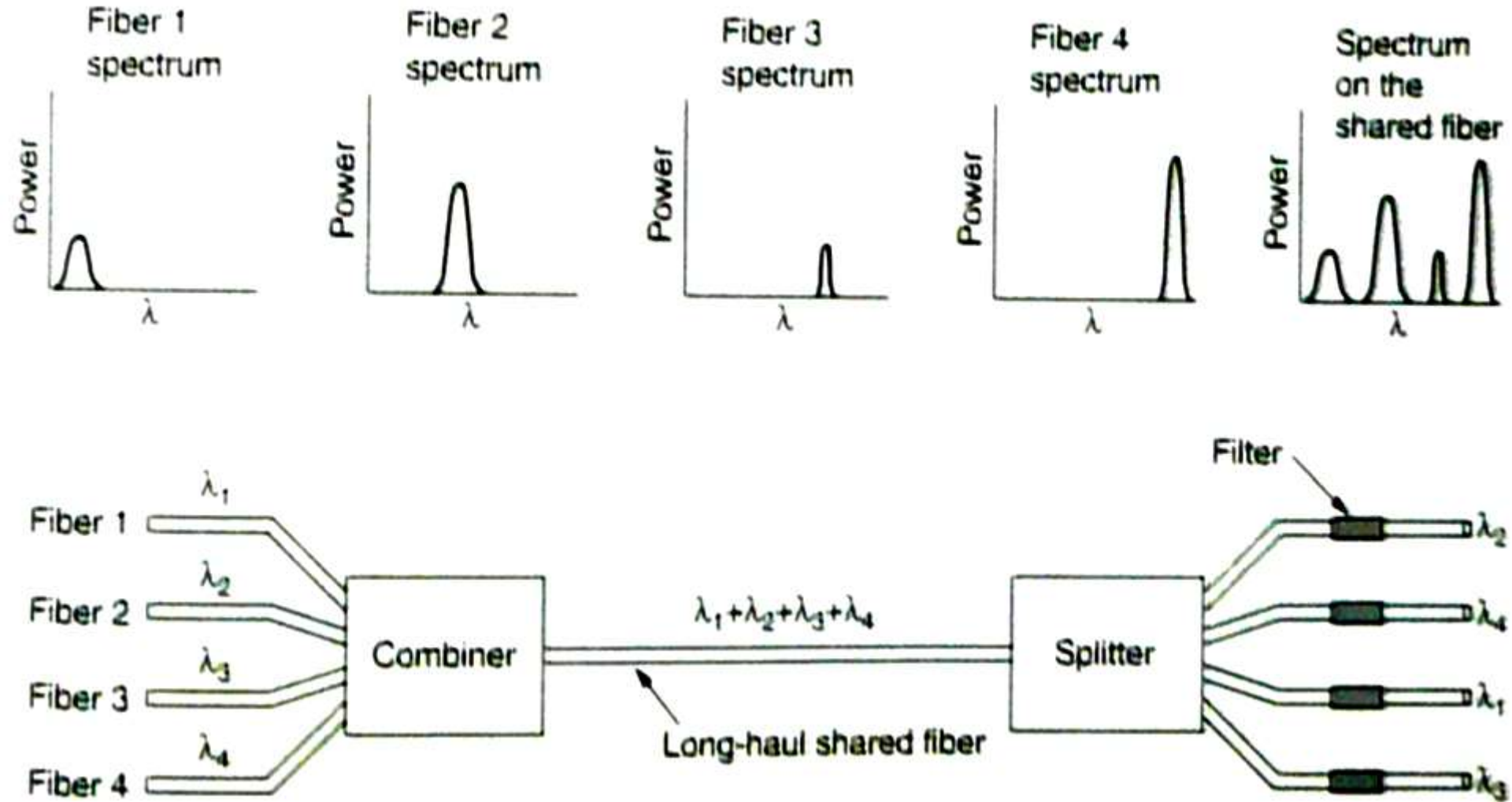


Frequency-division multiple access (FDMA)

- Separation of the whole spectrum into smaller frequency bands.
 - A channel gets a certain band of the spectrum for the whole time.
-
- + No dynamic coordination necessary.
 - + Works also for analog signals.
-
- As the frequency is reserved for certain user, it will lead to a waste of bandwidth if traffic is distributed unevenly.

Example: broadcast radio.

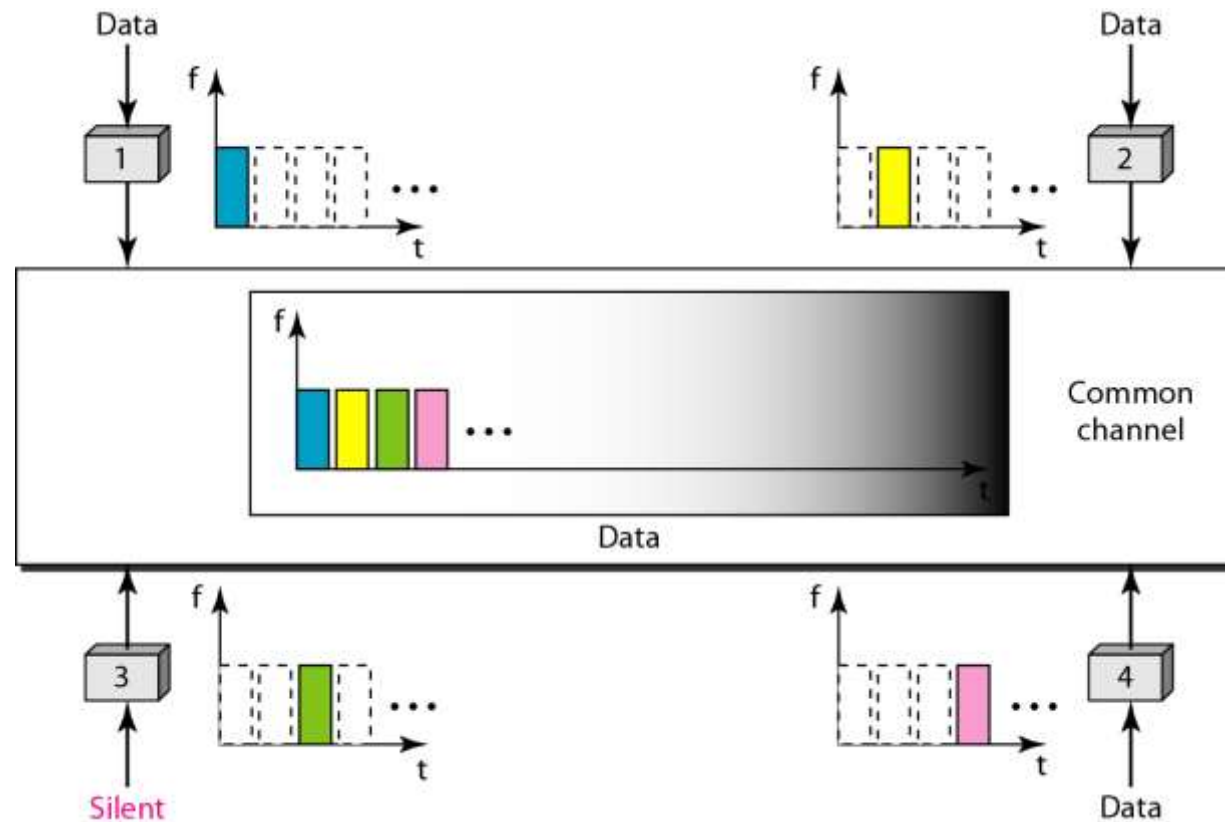
Wavelength Division Multiplexing (for optical fiber)



Time-division multiple access (TDMA) ★



- In TDMA, the bandwidth is just one channel that is timeshared between different stations

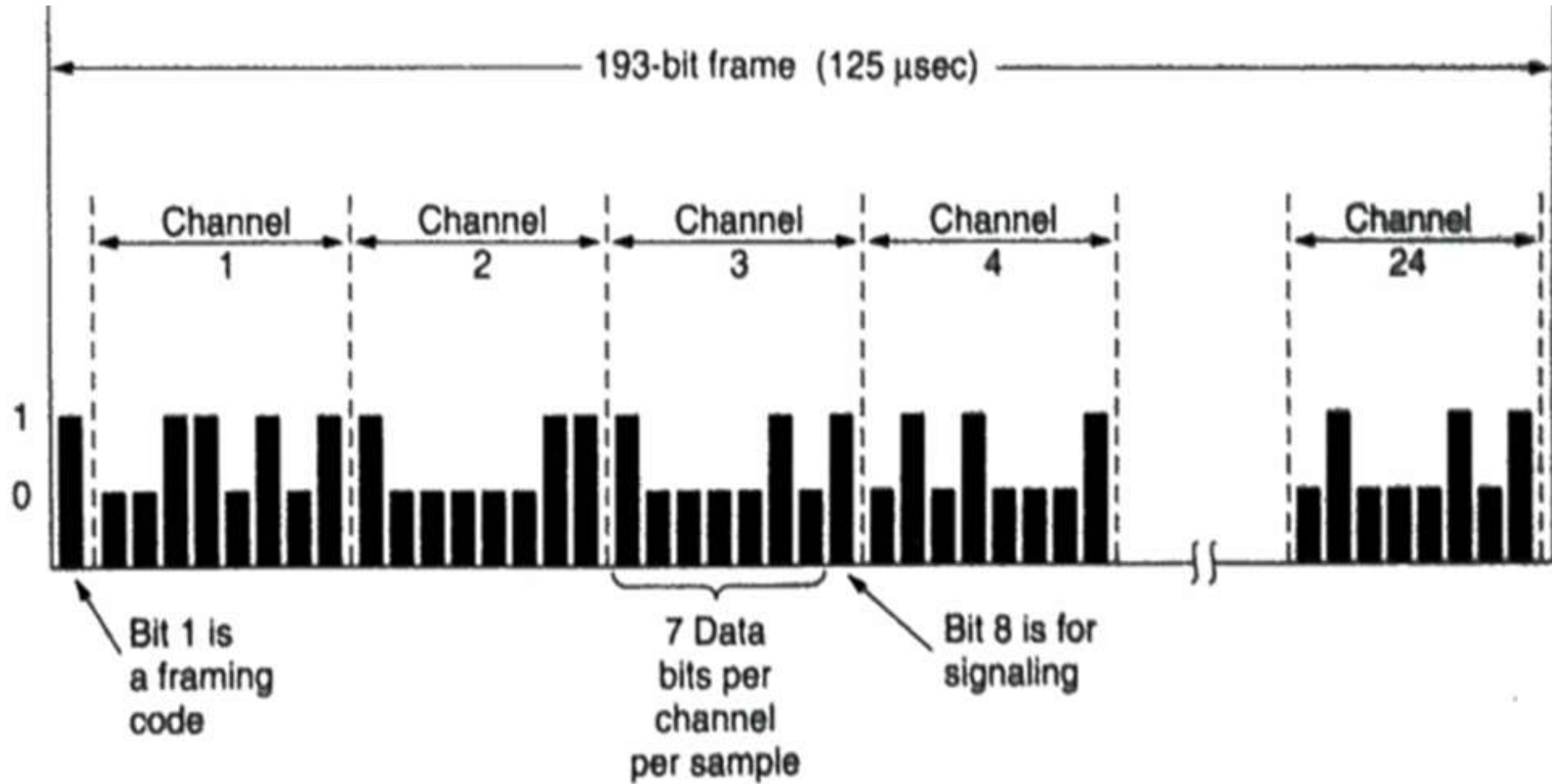


Time-division multiple access (TDMA)

- A channel gets the whole spectrum for a certain amount of time.
- + Only one carrier in the medium at any time.
- + Throughput high even for many users.
- Precise synchronization necessary. If two users are not synchronized, they may have collision when accessing the channel.

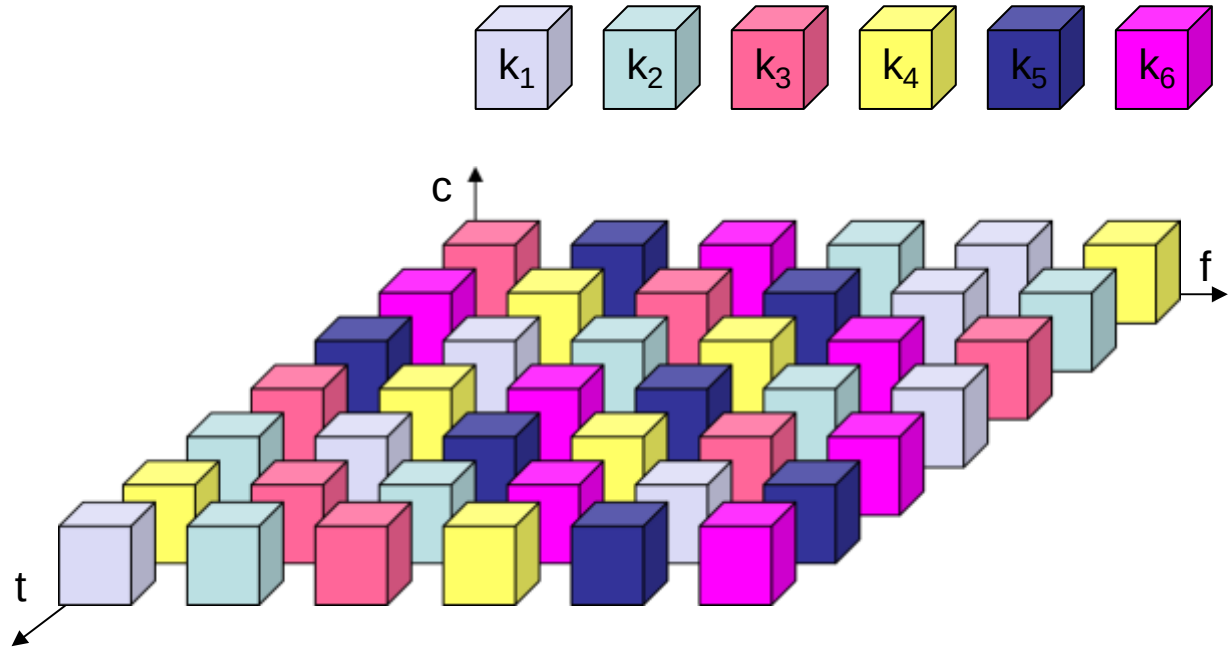
Example: Ethernet.

Time-division multiple access (TDMA)



Time and Frequency Division Multiplex

- Combination of both methods.
- A channel gets a certain frequency band for some time.
- Example: GSM (2G).



Code-division multiple access (CDMA)

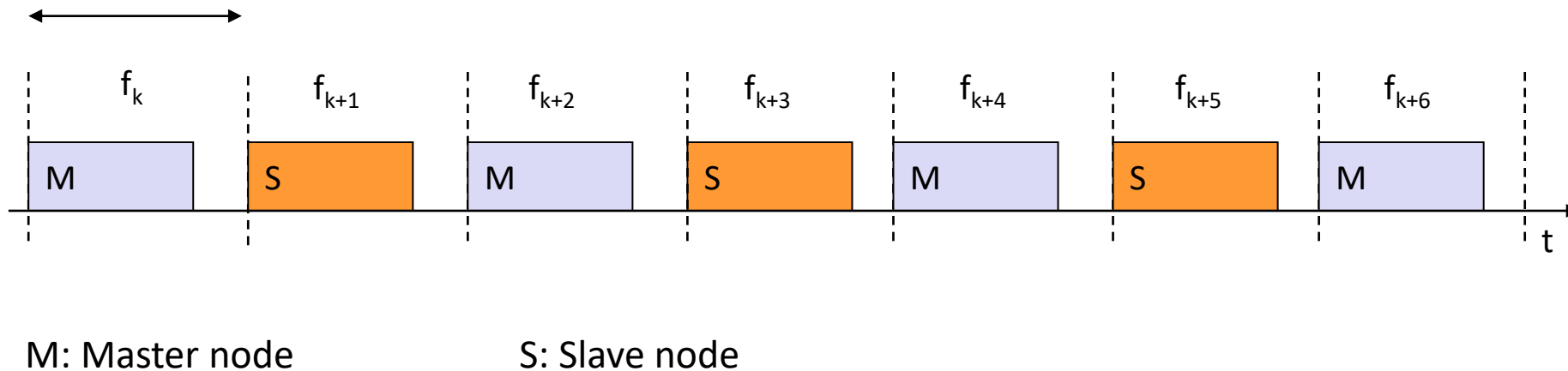


- ❑ In CDMA, one channel carries all transmissions simultaneously
- ❑ unique “code” assigned to each user; i.e., code set partitioning
 - all users share same frequency, but each user has own “chipping” sequence (i.e., code) to encode data
 - allows multiple users to “coexist” and transmit simultaneously with minimal interference (if codes are “orthogonal”)
- ❑ encoding: inner product: (original data) $\times$ (chipping sequence)
- ❑ decoding: summed inner-product: (encoded data) $\times$ (chipping sequence)

Frequency-hopping spread spectrum

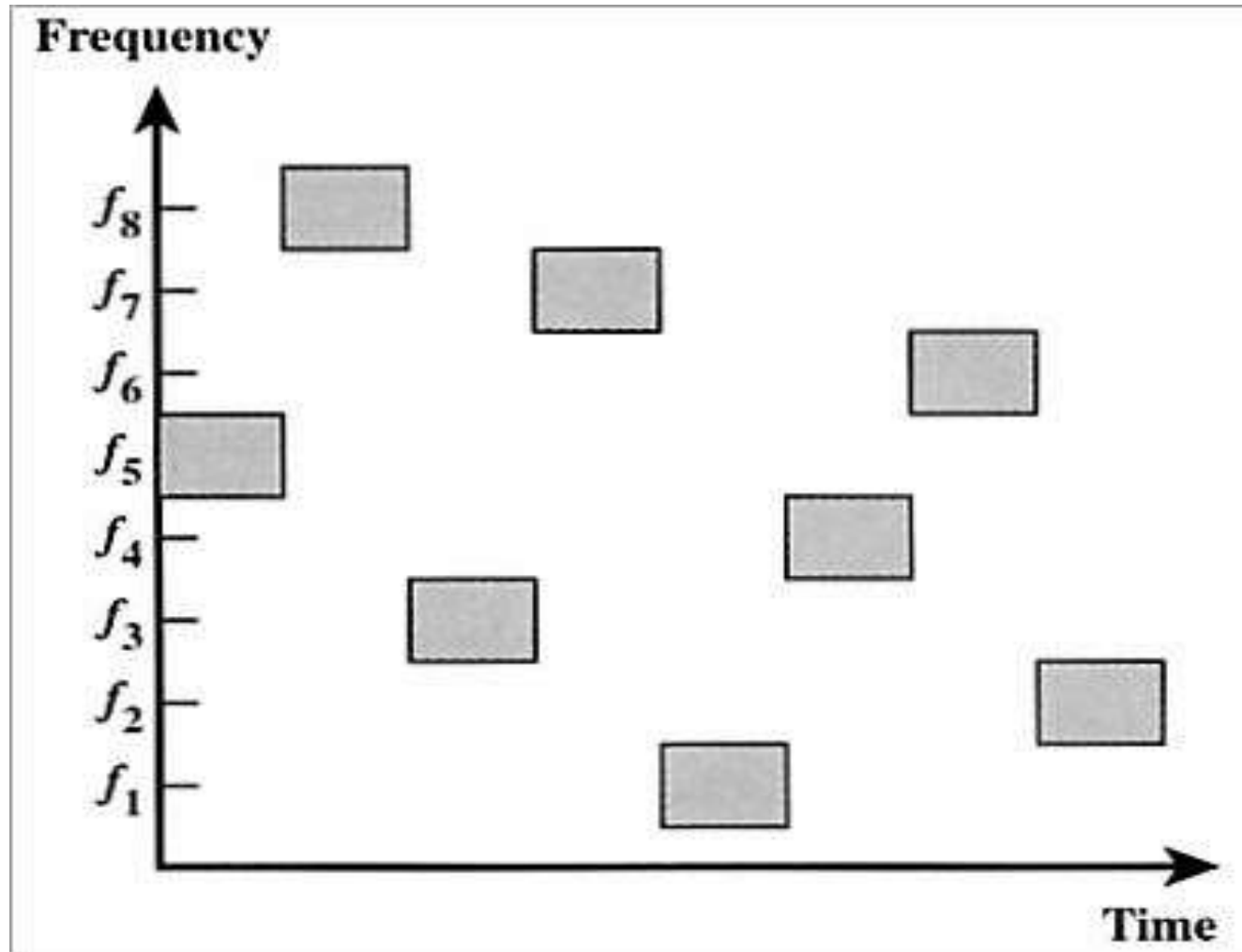
- The signal is broadcast over a seemingly random series of radio frequencies, hopping from frequency to frequency at fixed intervals.
- A receiver picks up the message by hopping between frequencies in synchronization with the transmitter.
- Steps
 - A number of channels are allocated for data transmission.
 - The transmitter operates in one channel at a time for a fixed interval. During that interval, some number of bits are transmitted.
 - The sequence of channels the transmitter used is dedicated by a spreading code.
 - The receiver uses the same code to receive the bits.

Example: Bluetooth Low Energy (BLE) ★



1. In a BLE network, the master node designs a unique frequency hopping pattern ($f_k, f_{k+1}, \dots, f_{k+6}$) and notifies the slave nodes within the network.
2. Master and slave nodes are then synchronized in clock.
3. Master and slave nodes hop together based on the agreed frequency hopping pattern.

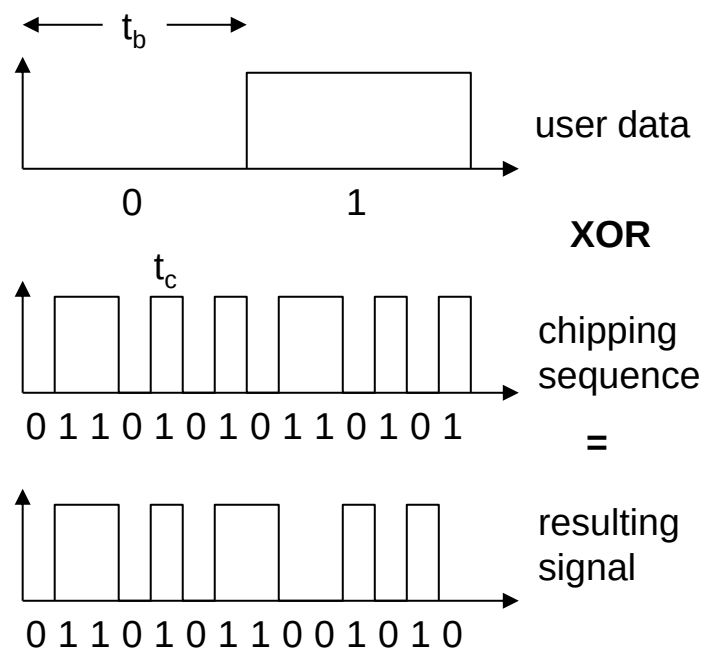
Frequency-hopping spread spectrum ★



Direct Sequence Spread Spectrum

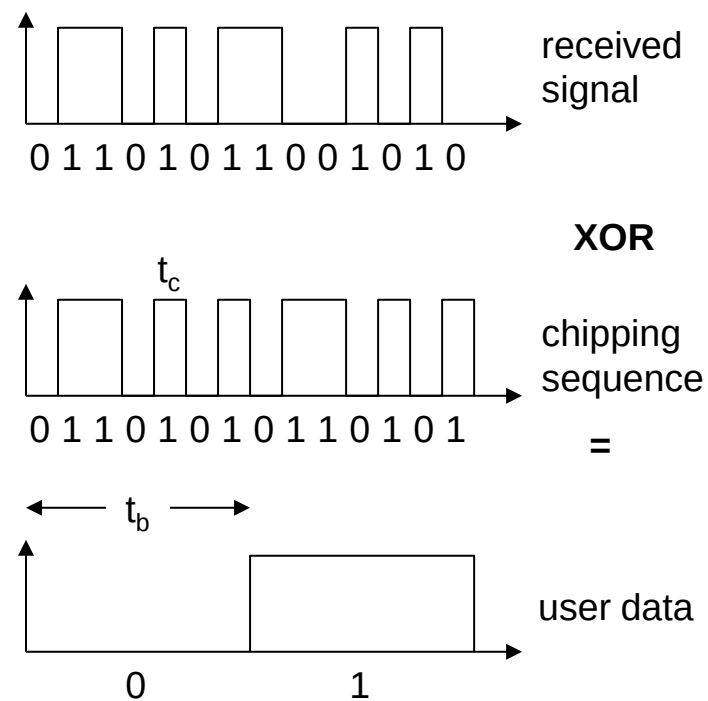
- Using the spreading code, each bit in the original signal is represented by multiple bits in the transmitted signal.
- The effect of the spreading code is to spread the signal across a wider frequency band.
- The receiver uses the same spreading code to recover the original signal.
 - Transmitting: the transmitter combines the digital information stream with the spreading code bit-stream using an exclusive-OR, and produces the transmitted signal. Then the signal is sent to the receiver.
 - Receiving: the receiver combines the received information stream with the same spreading code bit-stream using an exclusive-OR, then recovers the original information stream.

Example



Sender side

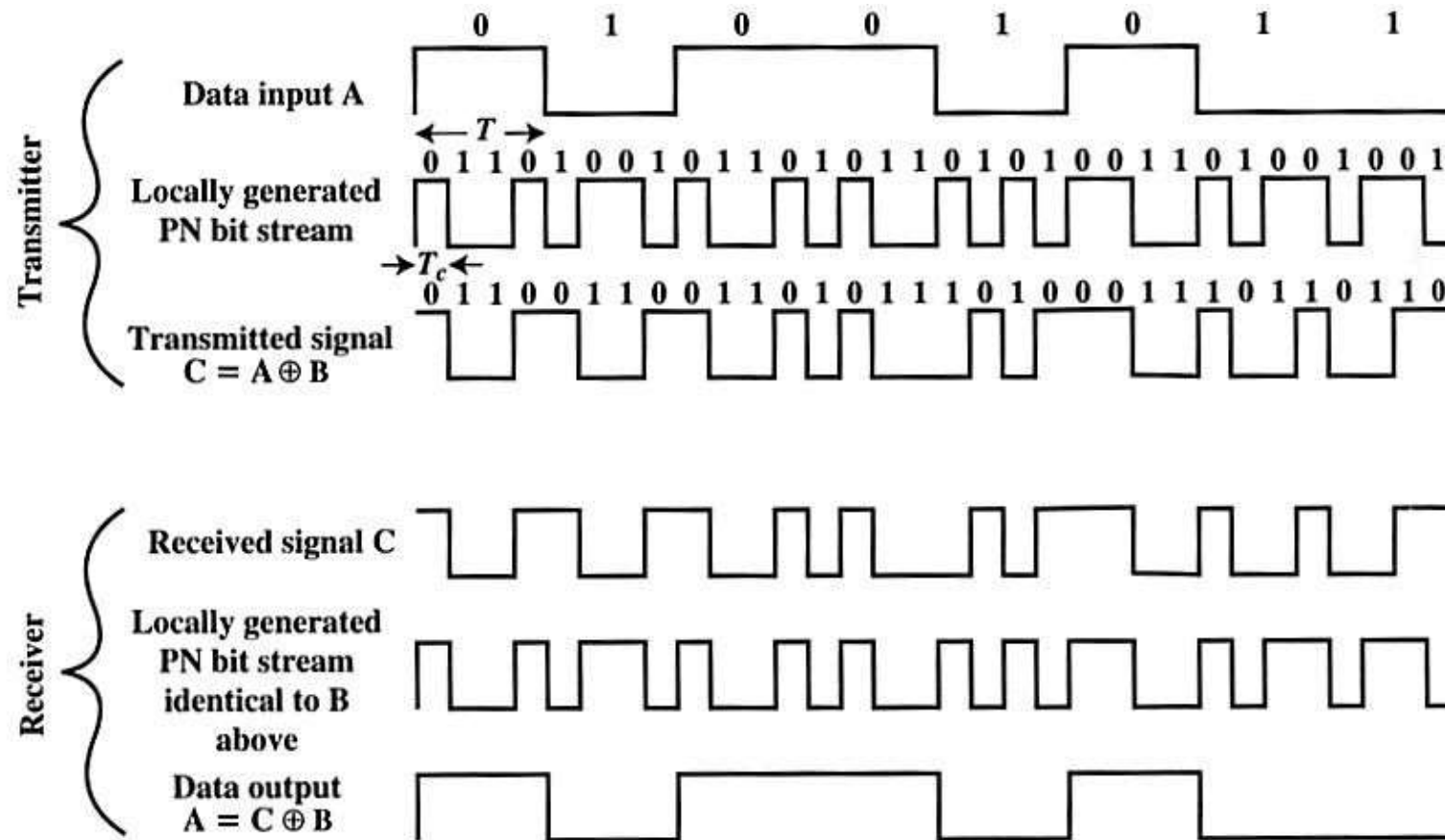
t_b : bit period



Receiver side

t_c : chip period

Direct Sequence Spread Spectrum



Random Access

- when node has packet to send
 - transmit at full channel data rate R
 - no *a priori* coordination among nodes
- two or more transmitting nodes: “collision”
- random access protocol specifies:
 - how to detect collisions
 - how to recover from collisions (e.g., via delayed retransmissions)
- examples of random access MAC protocols:
 - ALOHA, slotted ALOHA
 - CSMA, CSMA/CD, CSMA/CA

Random Access

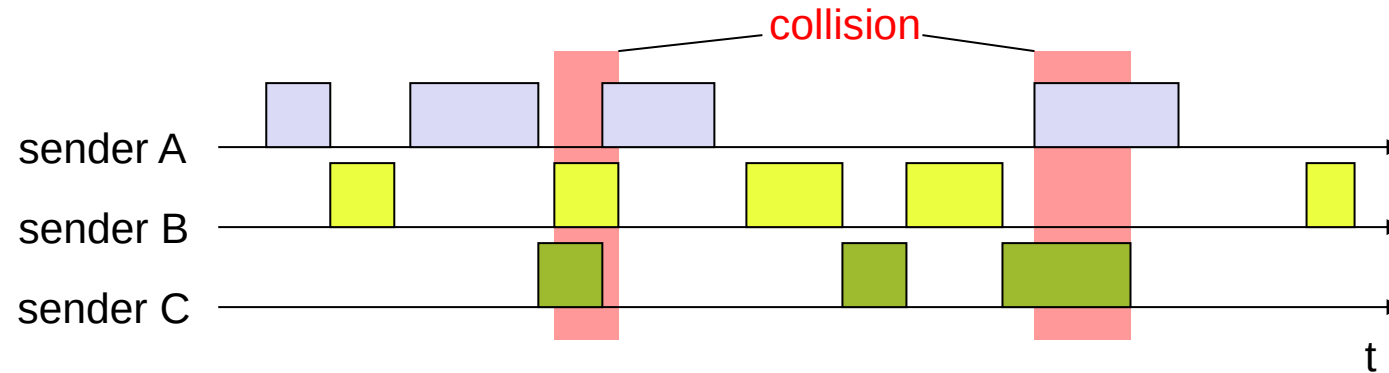
- In random access or contention methods, no station is superior to another station, and none is assigned the control over another.
- No station permits, or does not permit, another station to send.
- At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send.

Aloha/slotted aloha

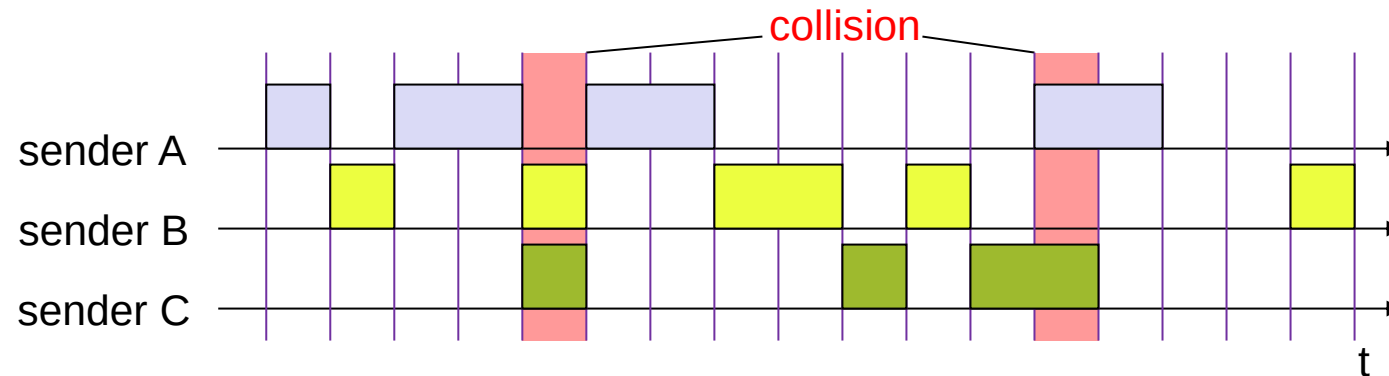
□ Mechanism

- Random, distributed (no central arbiter), time-multiplex
- Slotted Aloha additionally uses time-slots, sending must always start at slot boundaries

□ Aloha



□ Slotted Aloha



Aloha/slotted aloha

□ Aloha

- In pure Aloha, frames are transmitted at completely arbitrary times.

□ Slotted Aloha

- Time is divided into slots.
- A special station will be setup to provide synchronization signal.
- When a user has data to send, he/she waits until the beginning of the next slot.
- If there is a collision, the user retransmits the data again until successful.

Carrier Sense Multiple Access

❑ Persistent CSMA

- When a station has data to send, it first listens to the channel to see if anyone else is transmitting at that moment.
- If the channel is busy, the station waits until it is idle.
- When the station detects an idle channel, it transmits a frame.
- If a collision occurs, the station waits a random amount of time and start all over again.

❑ Non-persistent CSMA

- When a station has data to send, it first listens to the channel to see if anyone else is transmitting at that moment.
- If the channel is busy, it will wait for the random time and again sense the channel whether idle or busy.
- When the station detects an idle channel, it transmits a frame.

Ethernet CSMA/CD algorithm

1. Ethernet receives datagram from network layer, creates frame
2. If Ethernet senses channel:
 - if **idle**: start frame transmission.
 - if **busy**: wait until channel idle, then transmit
3. If entire frame transmitted without collision - done!
4. If another transmission detected while sending: abort, send jam signal
5. After aborting, enter *binary (exponential) backoff*:
 - after m th collision, chooses K at random from $\{0, 1, 2, \dots, 2^m - 1\}$. Ethernet waits $K \cdot 512$ bit times, returns to Step 2
 - more collisions: longer backoff interval